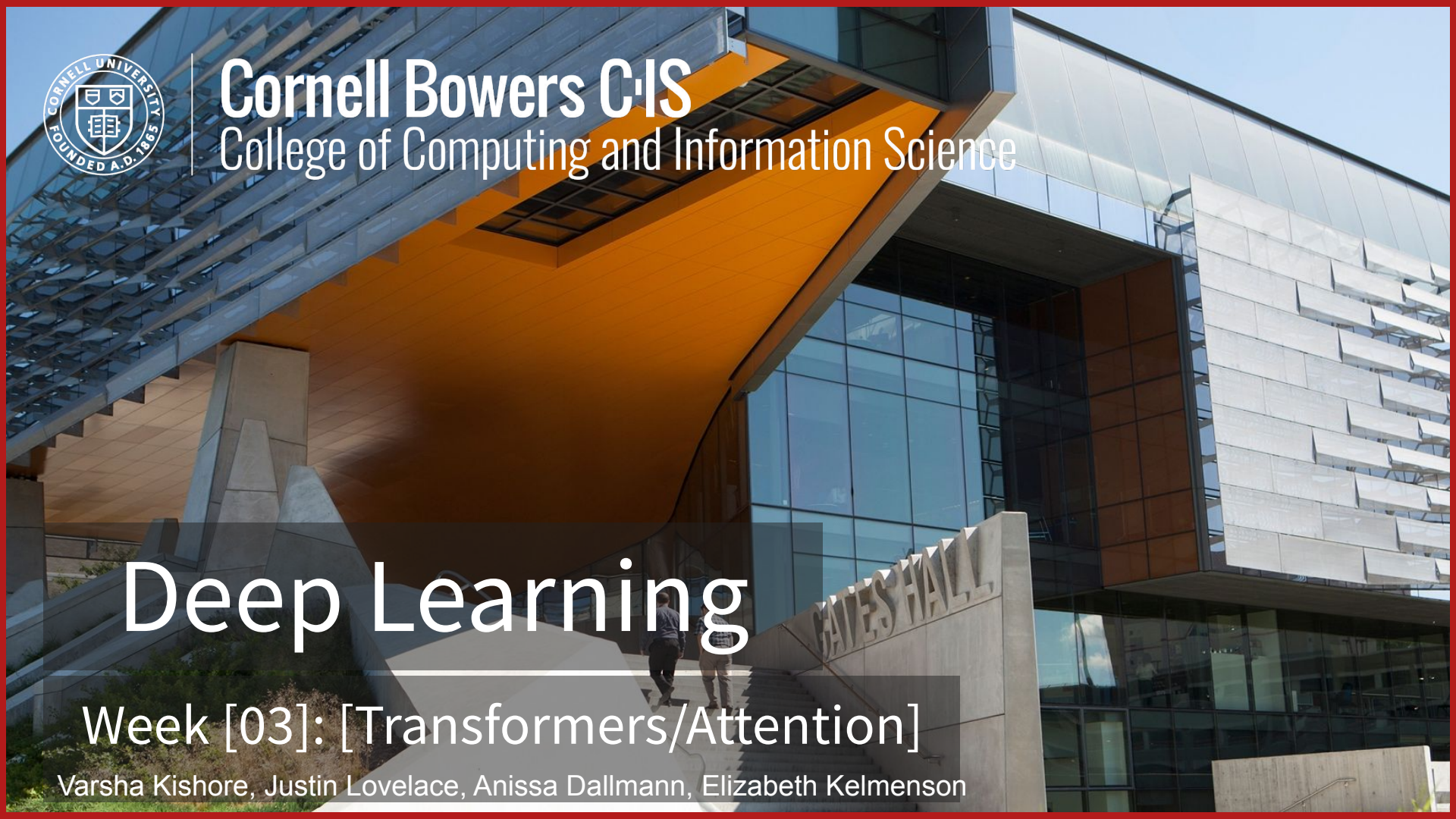


Deep Learning

Week [03]: [Transformers/Attention]

Varsha Kishore, Justin Lovelace, Anissa Dallmann, Elizabeth Kelmenson

Logistics

- HW2 due 03/03
- A2 due 03/03
- HW3 is out 03/03, due 03/12
- Project Proposal (see next three slides) due 03/07

Project

- **Aim:** to get hands on experience with implementing modern deep learning methods
- Find a recent deep learning research paper that was published in the last 1-5 years (a list of approved papers will also be provided to choose from)
- Reproduce a specific result from the paper
- To be completed in groups of 2-4

Project Grading

- Project proposal
  - We want to make sure the papers you've chosen are reasonable
    - Possible on Collab (GPU limits)
  - Sign up on spreadsheet, we don't want duplicates!
- Final presentation
  - The last 2-3 classes will be devoted to presentations
  - Present the paper you've chosen to the class
  - Discuss whether you were able to replicate results from the paper and describe any obstacles
- Final deliverable
  - Github repo with re-implementation
  - Readme with method description, instructions to run the code and results
  - More detail will be provided later

Project Proposal

A page long project proposal due **March 7**. It should contain the following:

- Paper selection:
  - Title, authors, and publication venue of the chosen paper
  - Brief summary of the chosen paper
  - Brief justification of why you choose this paper
- Result Selection
  - Tell us which result you want to replicate
- Re-implementation Plan
  - Describe architecture, method, and metrics
  - Details about how much compute and time is required to replicate results
- **Detailed instruction will be provided on Canvas/Ed Discussion**

What you will learn today

- Issues with LSTMs
  - Meaning of words flows both ways (fixable)
  - Bottleneck layers (fixable)
  - LSTMs are slow (not fixable)
- Attention:
  - Scaled Inner-product attention
  - Self-Attention
  - Masked self-attention
  - Multi-head attention
  - Cross-attention
- New Concepts:
  - Positional Embedding
  - Layer Norm
- New Algorithms:
  - ELMO
  - Transformer
  - BERT

Cornell Bowers CIS

LSTMs

- Add a cell state to store information
  - Gradient flows along the cell state
- Update cell state with parameterized gating functions
- Performs better with long sequences

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

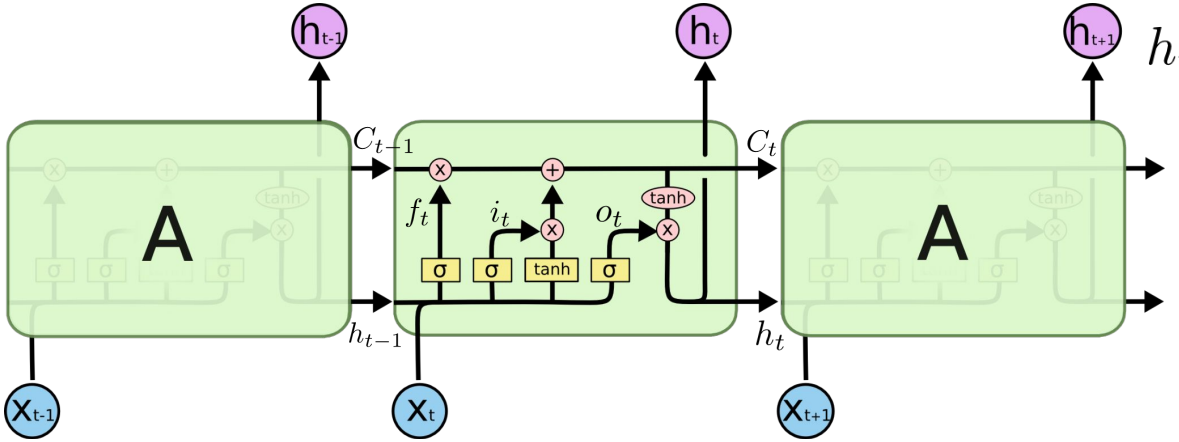
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$o_t = \sigma\left(W_o \; [h_{t-1}, x_t] \; + \; b_o\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

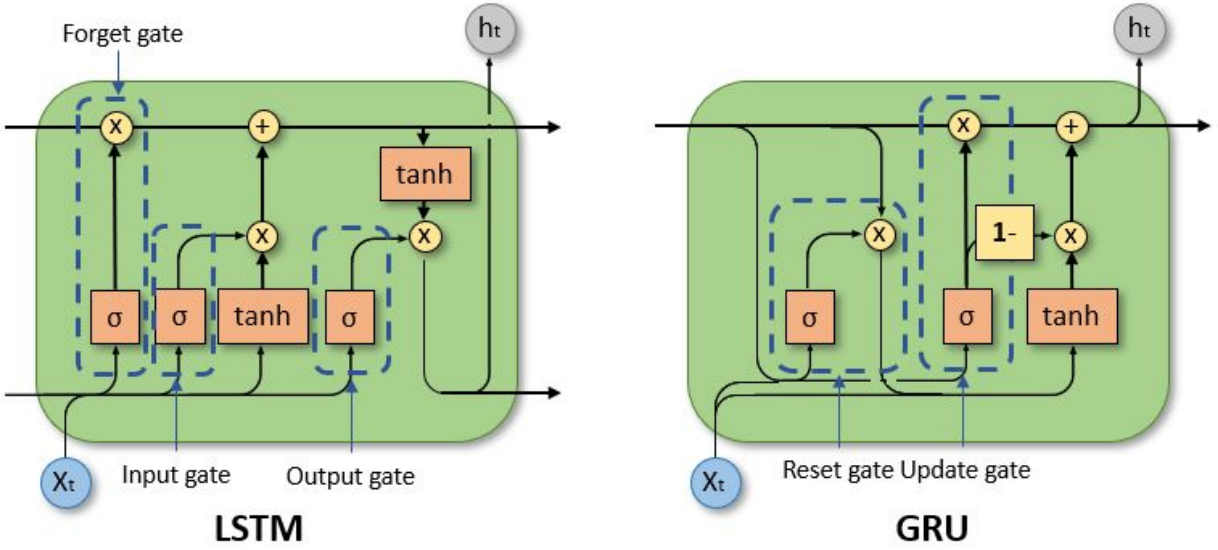
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$h_t = o_t * \tanh\left(C_t\right)$$



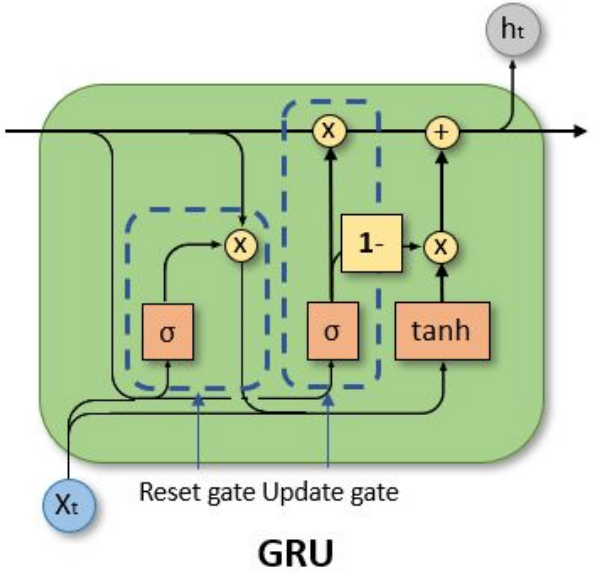
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Gated recurrent units (GRUs)



https://towardsdatascience.com/illustrated-guide-to-lstms-and-gru-s-a-step-by-step-explanation-44e9eb85bf21

Gated recurrent units (GRUs)

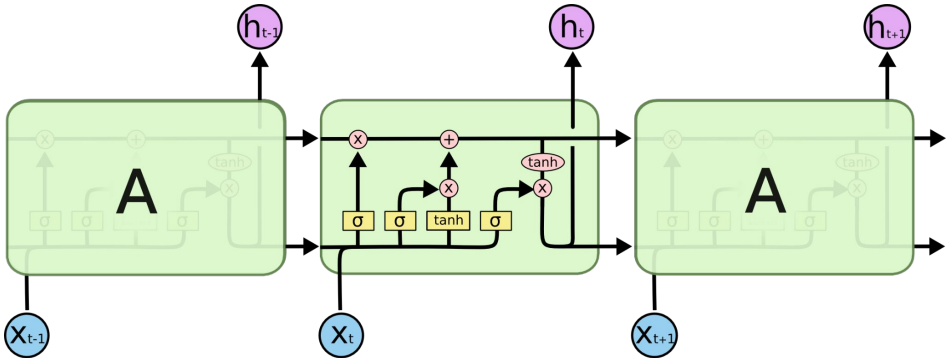


GRU

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$
$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$
$$\hat{h}_t = \phi(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$$
$$h_t = (1 - z_t) \odot \hat{h}_t + z_t \odot h_{t-1}$$

Previously: Using LSTMs to solve sequence problems

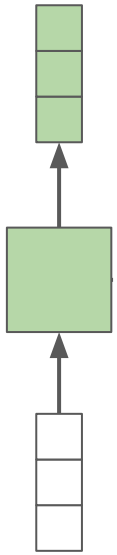
- Process sequences one element at a time.
- Maintain a 'memory' (cell state) to capture information about previous steps.
- Mitigates the RNN vanishing gradient problem
- Suitable for time series, speech, text, and other sequential data.



Autoregressive Generation

With LSTMs or RNNs

Machine Translation

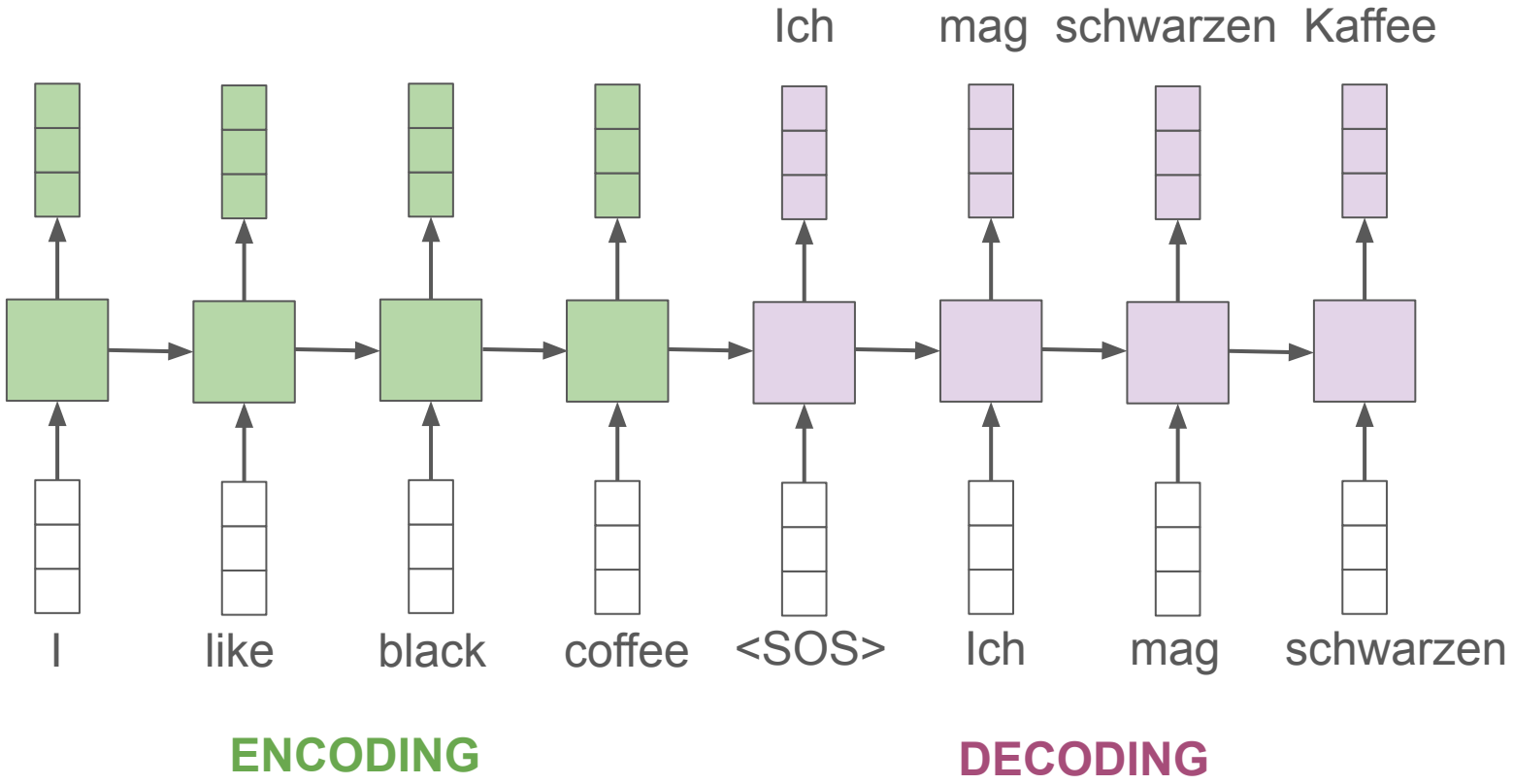


I like black coffee

ENCODING

DECODING

Machine Translation



Issue #1

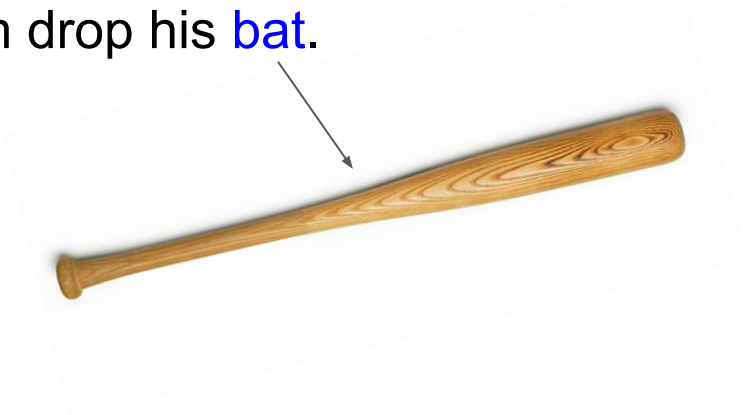
Context influences words in both directions

Context influences word meaning.

A bat flew out of the dugout, startling the baseball player and making him drop his bat.

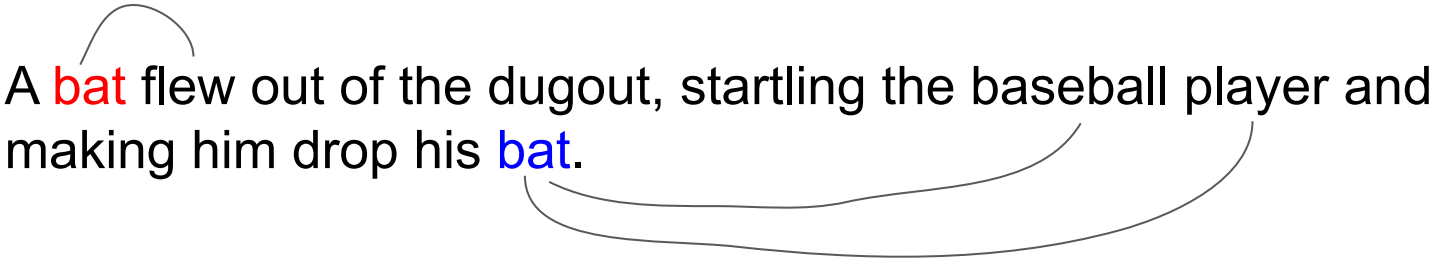
Context influences word meaning.

A bat flew out of the dugout, startling the baseball player and making him drop his bat.



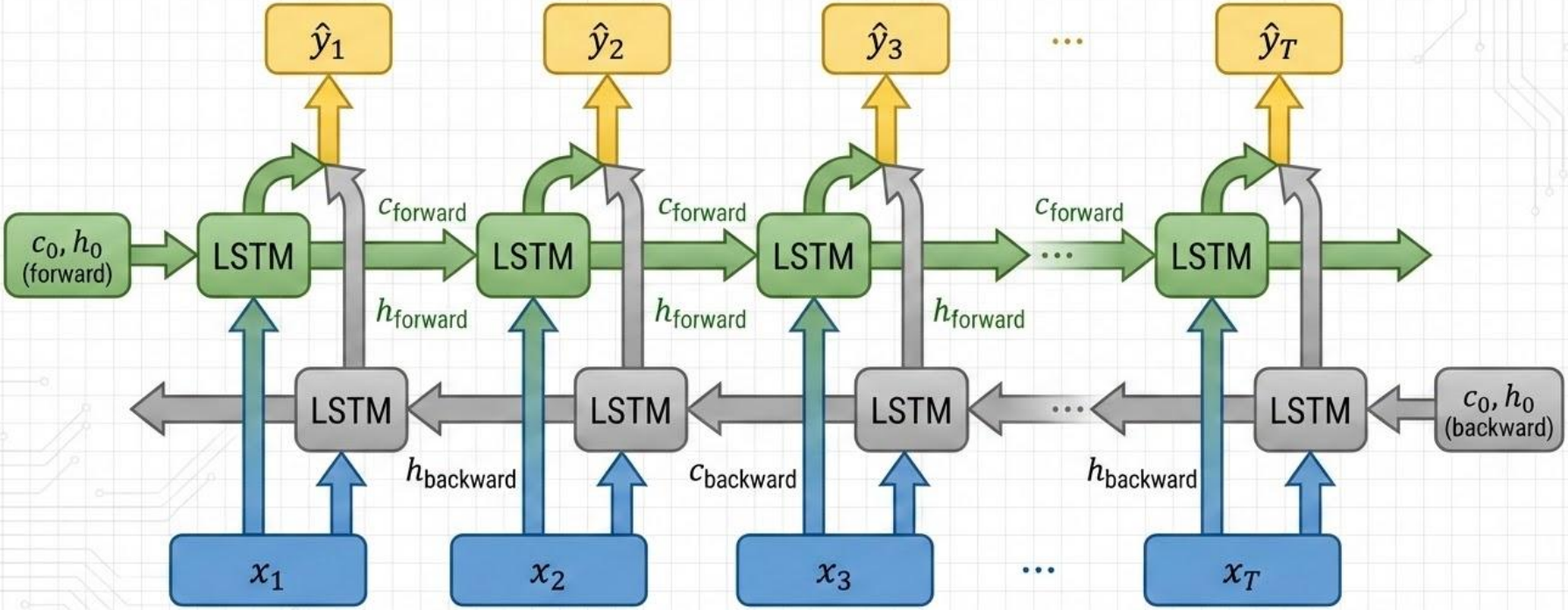
Context influences word meaning.

Each word pays attention to the words around it, focusing more on the words which provide important information about its meaning.



Cornell Bowers CIS

Bidirectional LSTM



Cornell Bowers CIS

Bidirectional LSTM

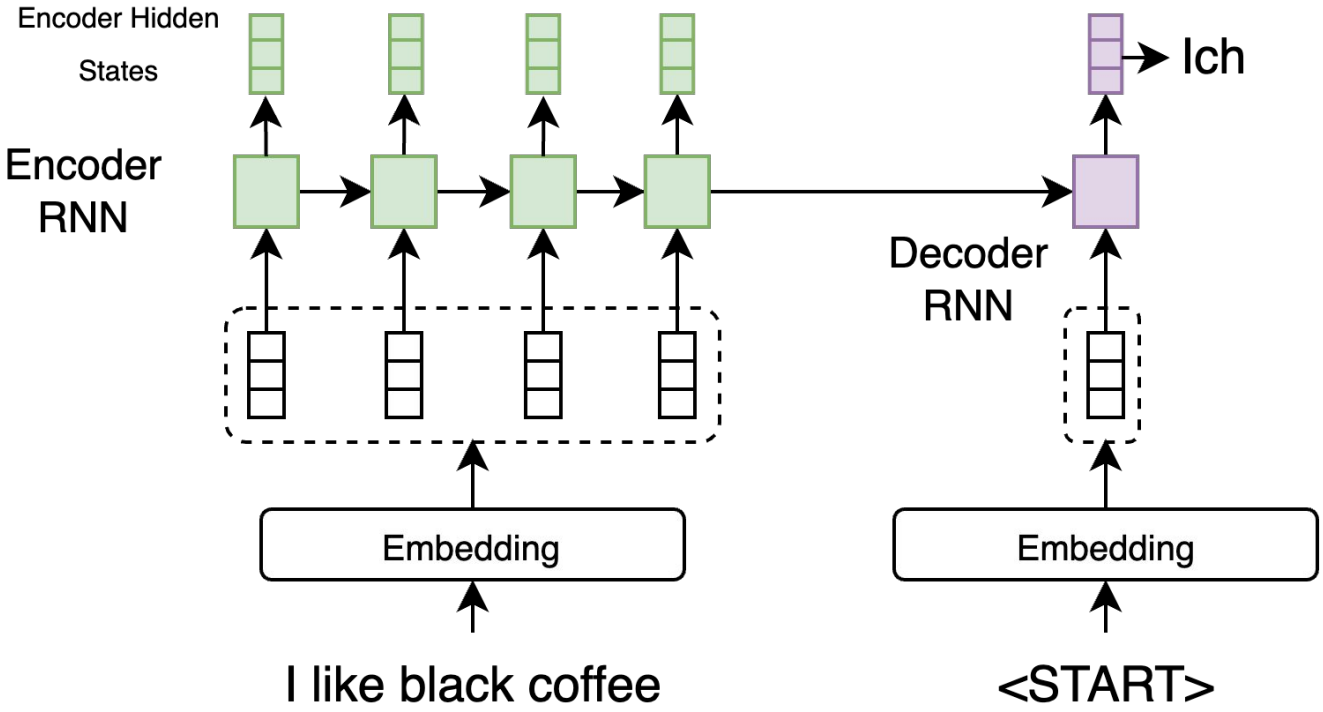
The output vectors are **CONTEXTUAL WORD EMBEDDINGS!**

Embeddings from Language MOdel (ELMO)
[Peters et al. 2018]

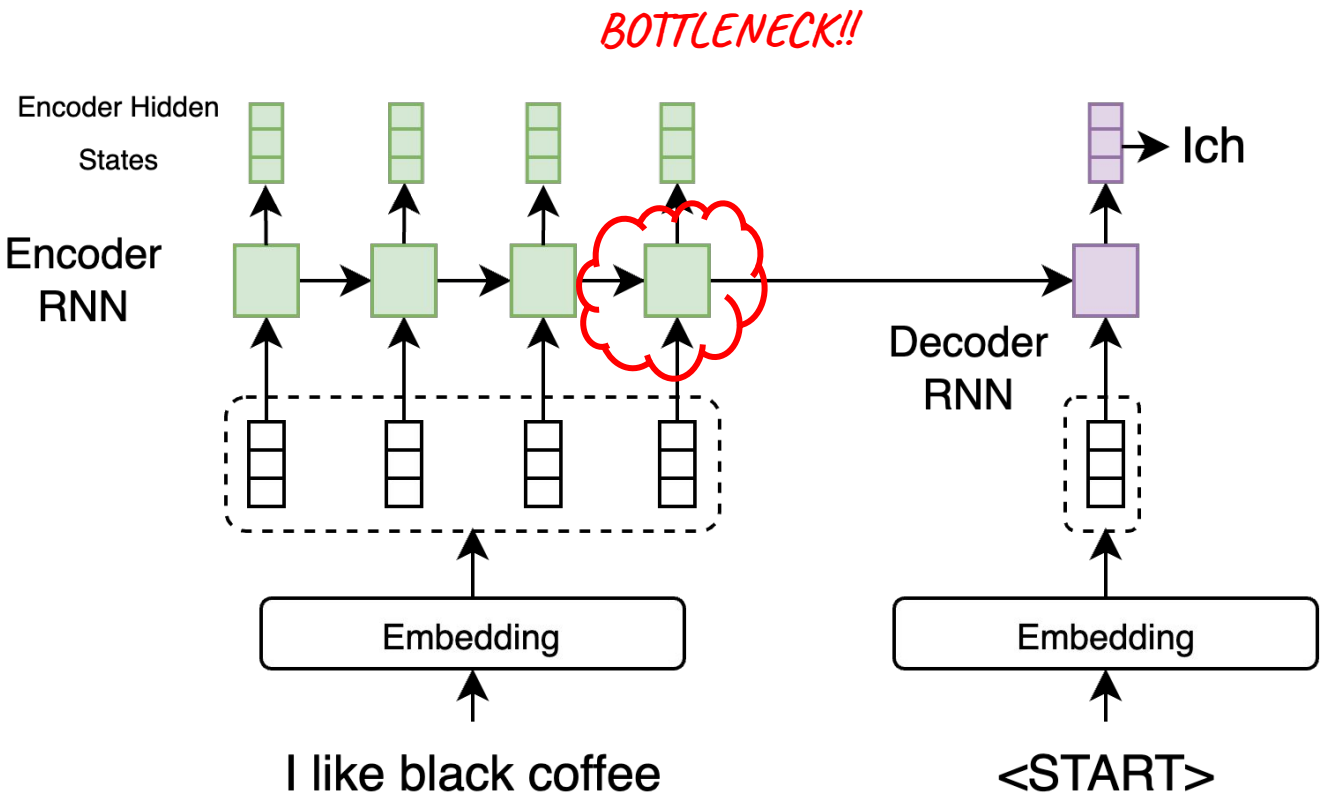
Issue #2

The Bottleneck!

RNN for Machine Translation

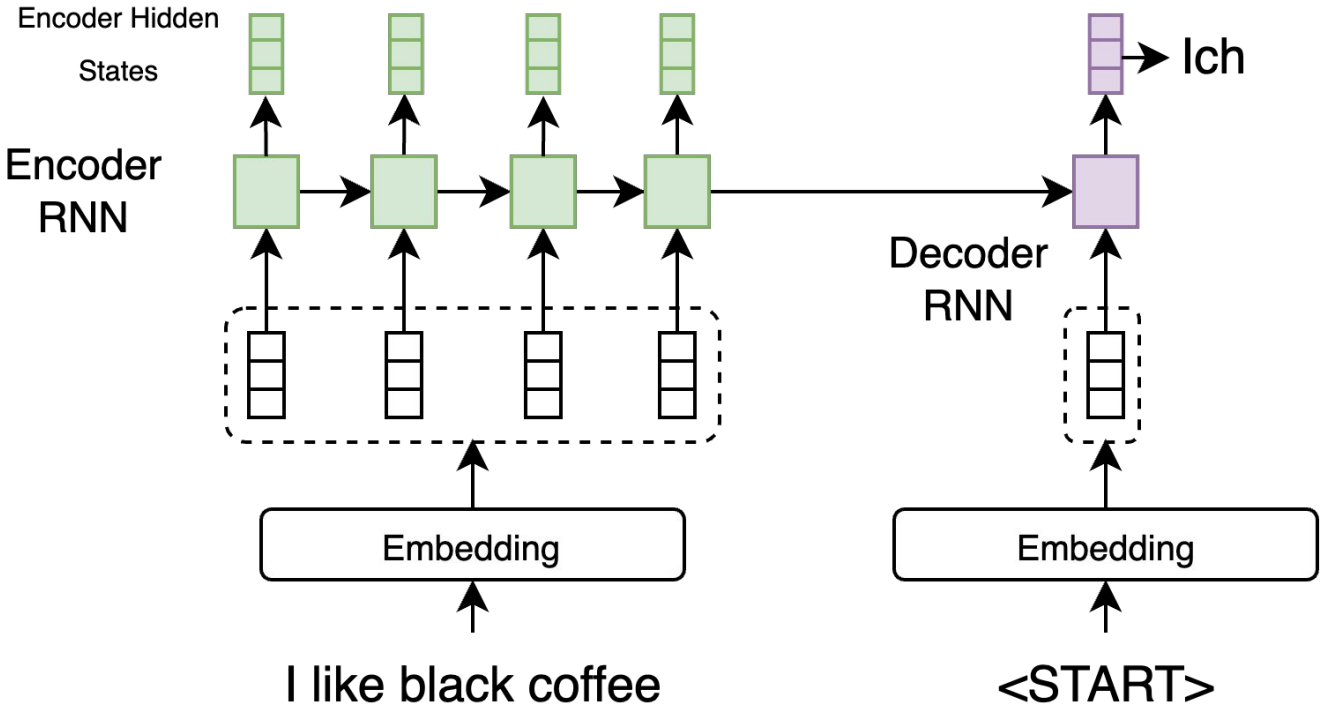


Bottleneck Problem



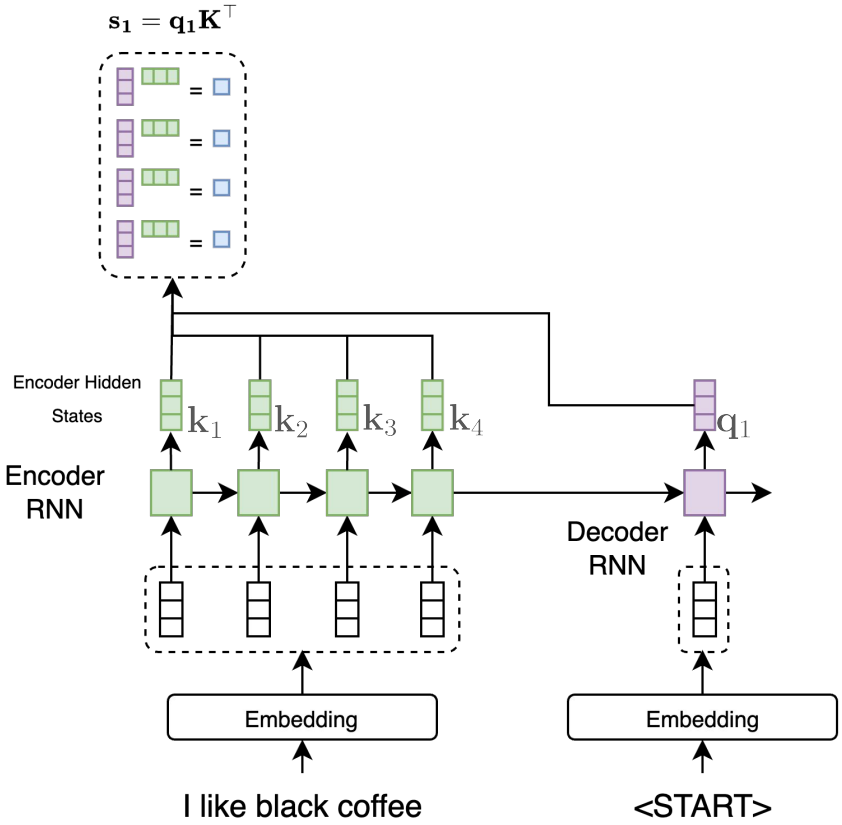
RNN for Machine Translation

Would be nice if we could “look back” at previous hidden states



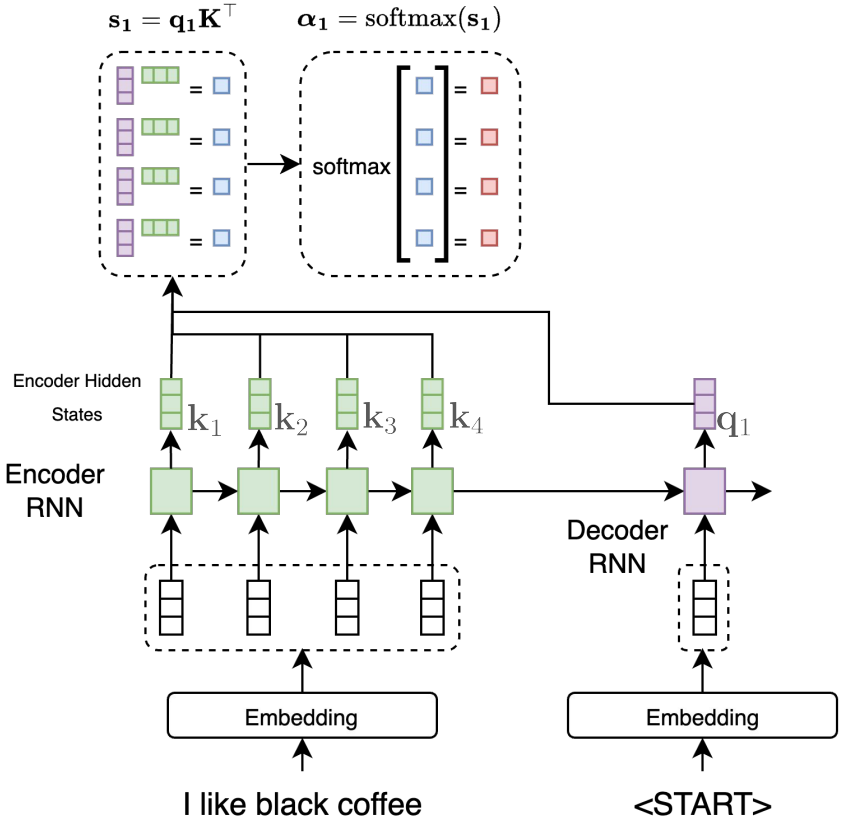
RNN with Attention

[Bahdanau et al. 2014]



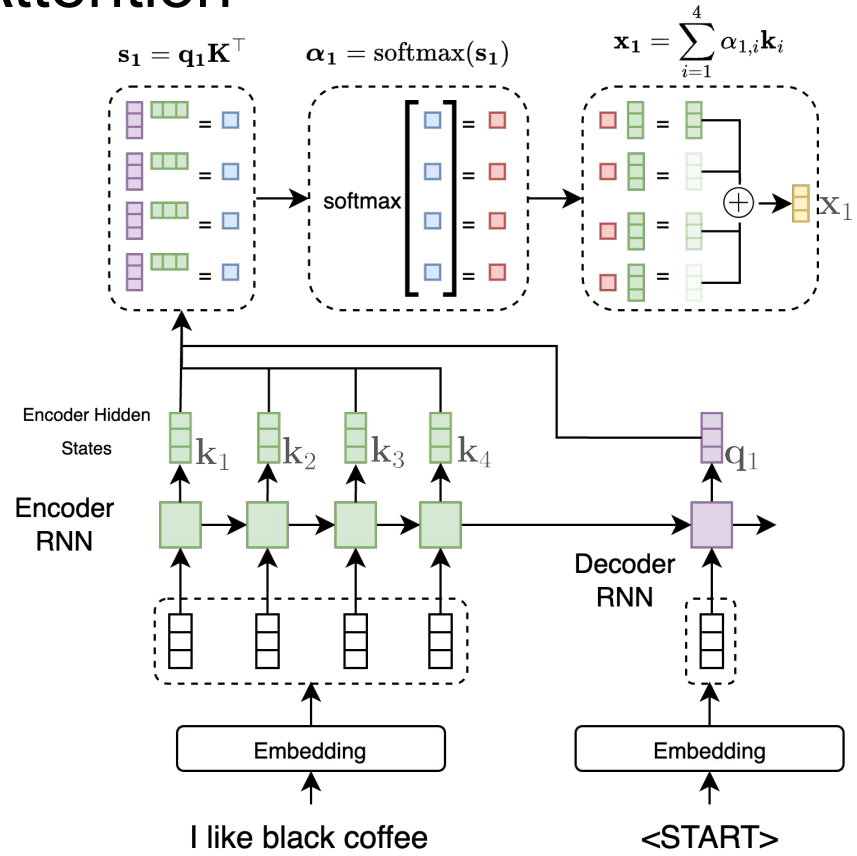
RNN with Attention

[Bahdanau et al. 2014]



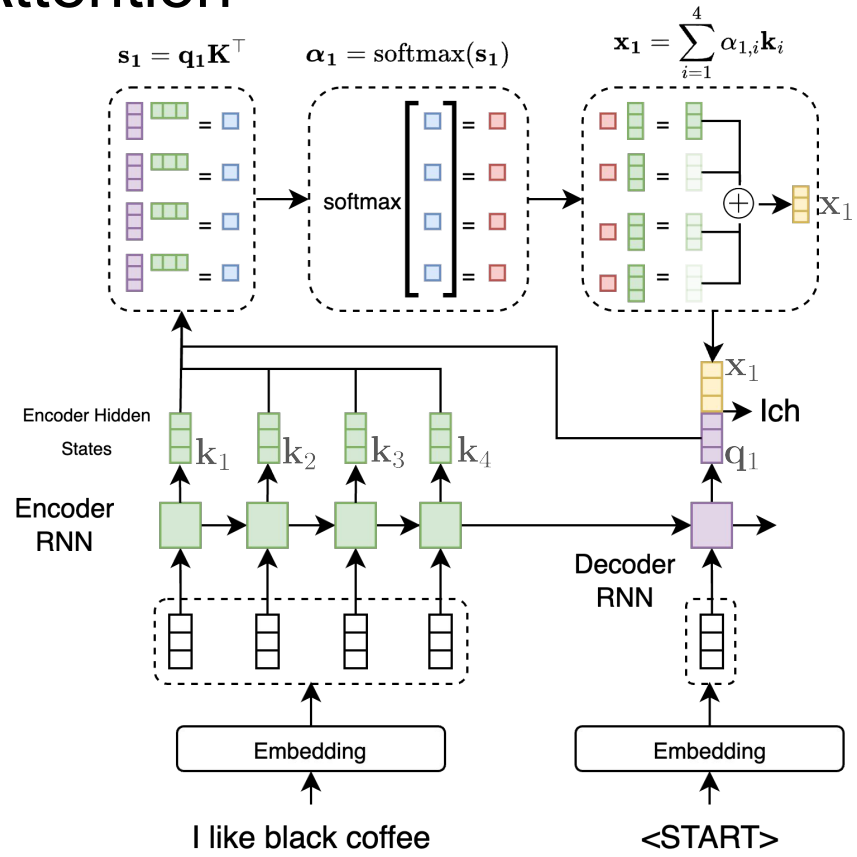
RNN with Attention

[Bahdanau et al. 2014]



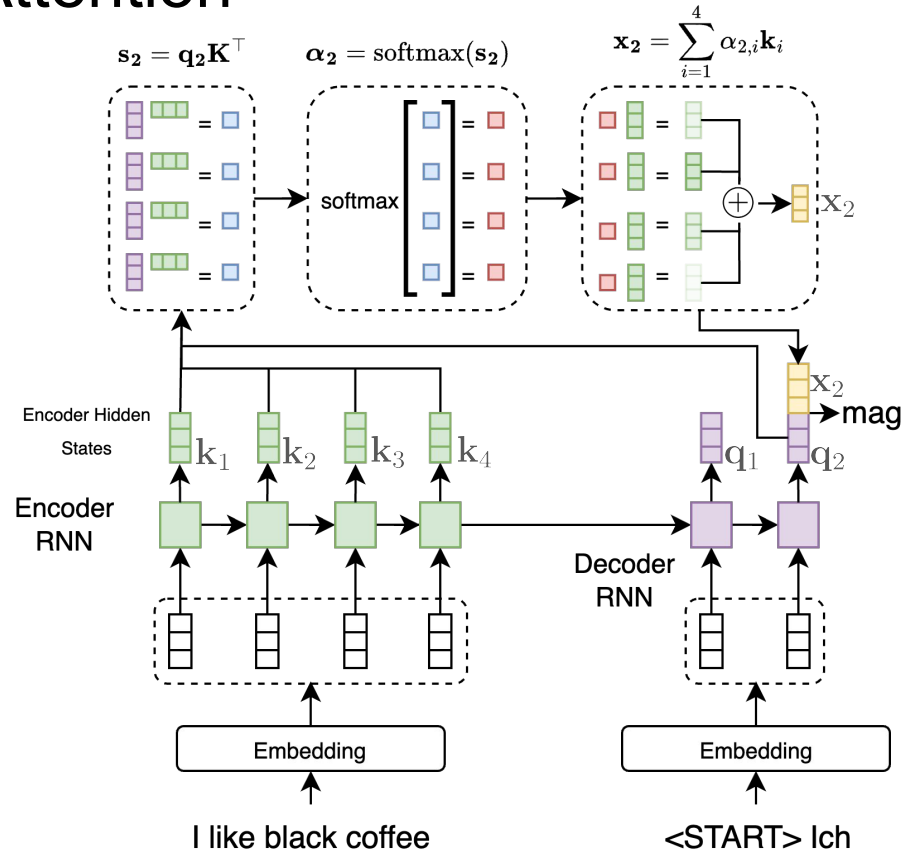
RNN with Attention

[Bahdanau et al. 2014]



RNN with Attention

[Bahdanau et al. 2014]



Issue #3

Recurrent Neural Networks (including LSTMs) are slow on modern computers (with GPUs). Why? - Discuss

Cornell Bowers CIS

TRANSFORMERS [Vaswani et al. 2017]

- **Parallel** alternative to LSTMs
- Far more **efficient** on GPUs
- **Removes recurrence**
- **Keeps attention**
- Currently dominant approach for most sequence tasks
  - E.g. Machine translation, summarizatior question answering, etc.

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez*† University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks in an encoder-decoder configuration. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

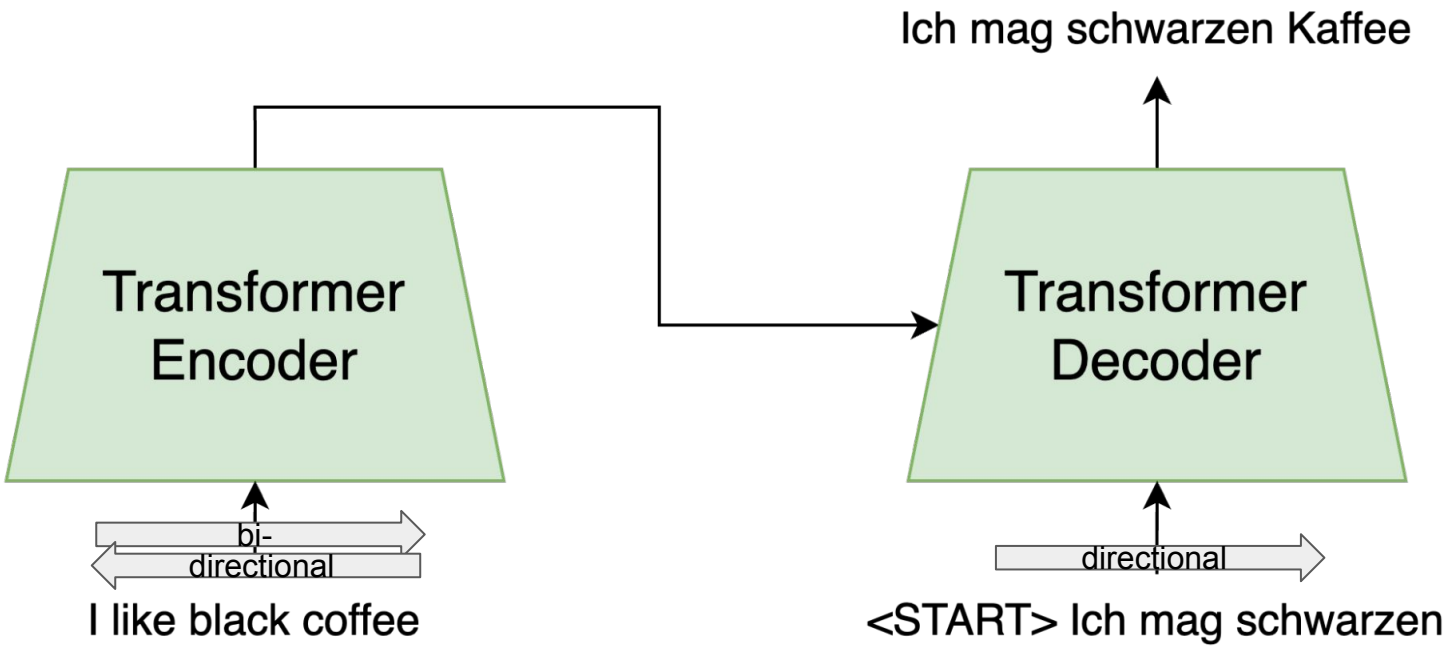
1 Introduction

Recurrent neural networks, long short-term memory [12] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [31, 2, 5]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [34, 22, 14].

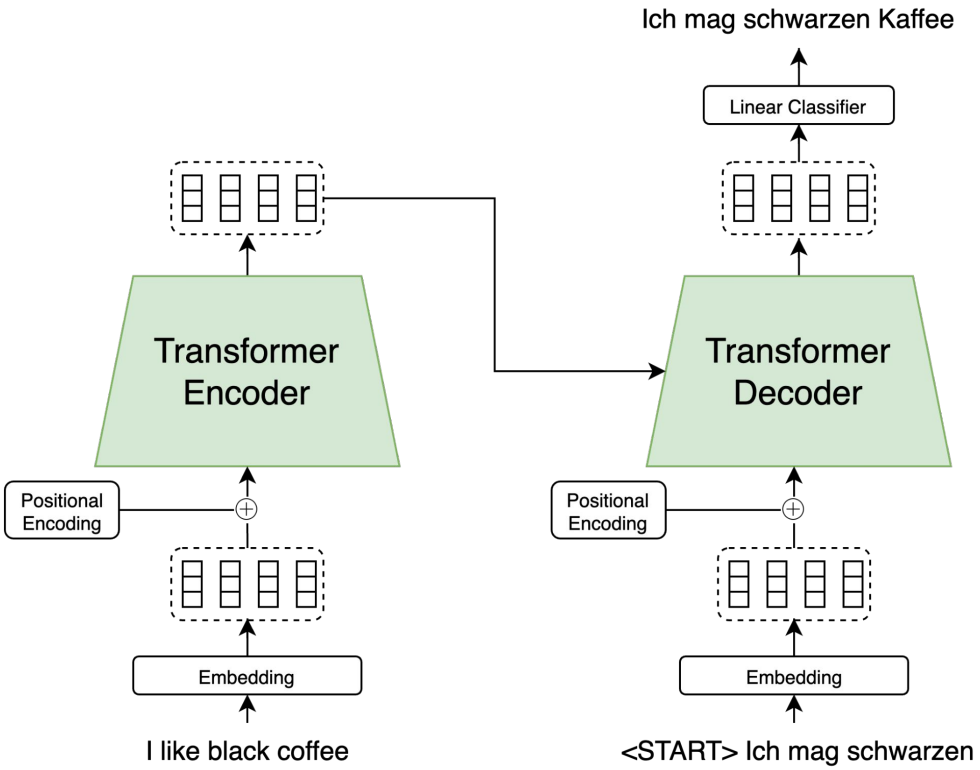
Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states h_t, as a function of the previous hidden state h_{t-1} and the input for position t. This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks [19] and conditional

*Equal contribution. Listing order is random.
†Work performed while at Google Brain.

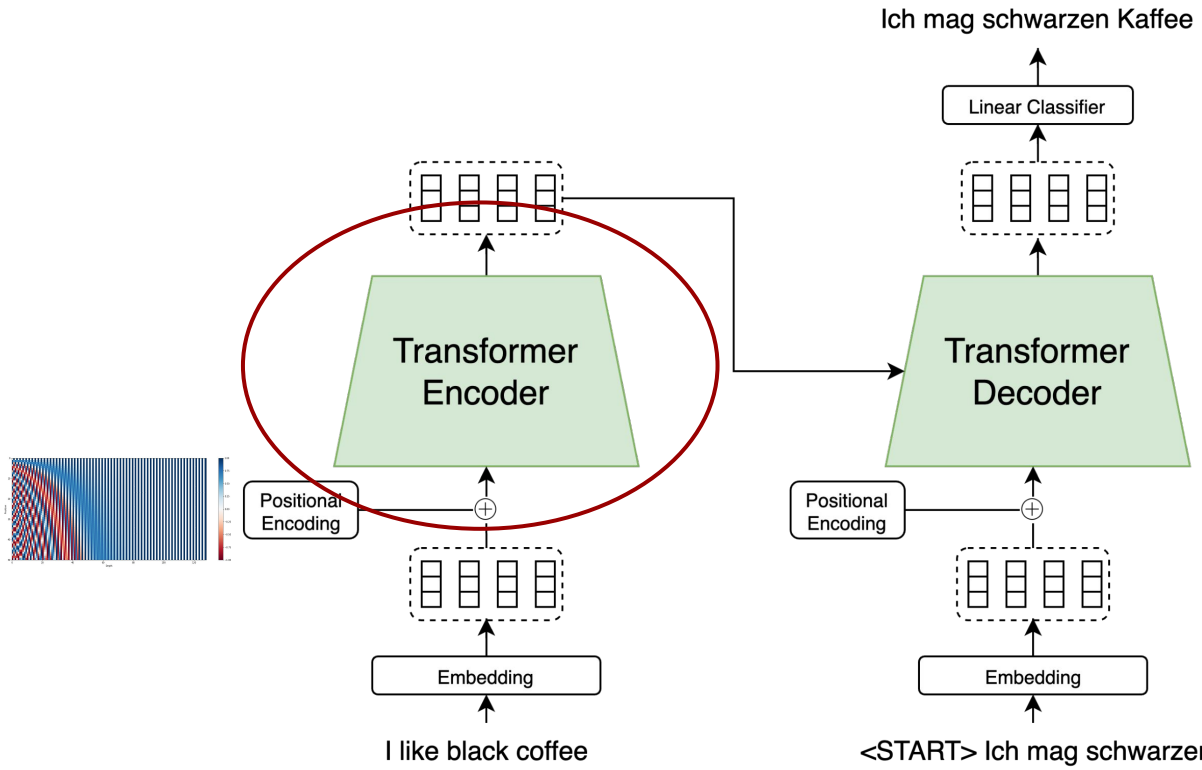
Transformer Architecture



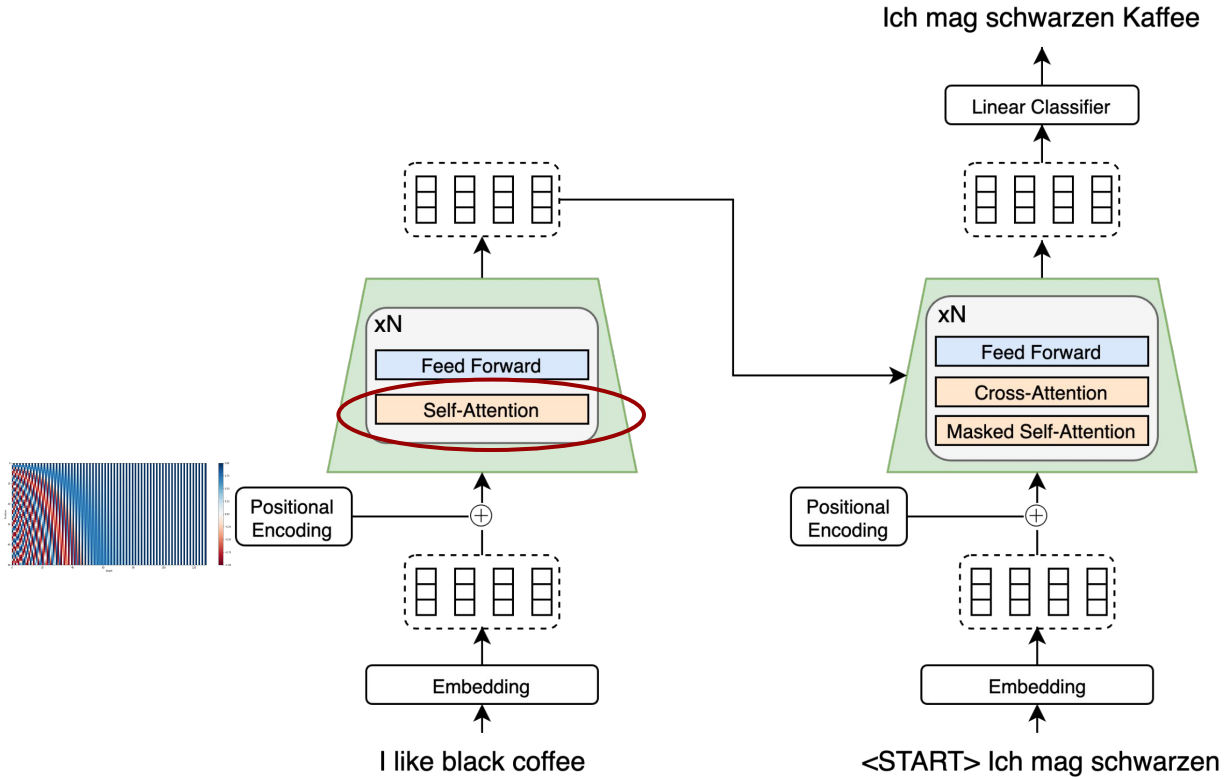
Transformer Architecture



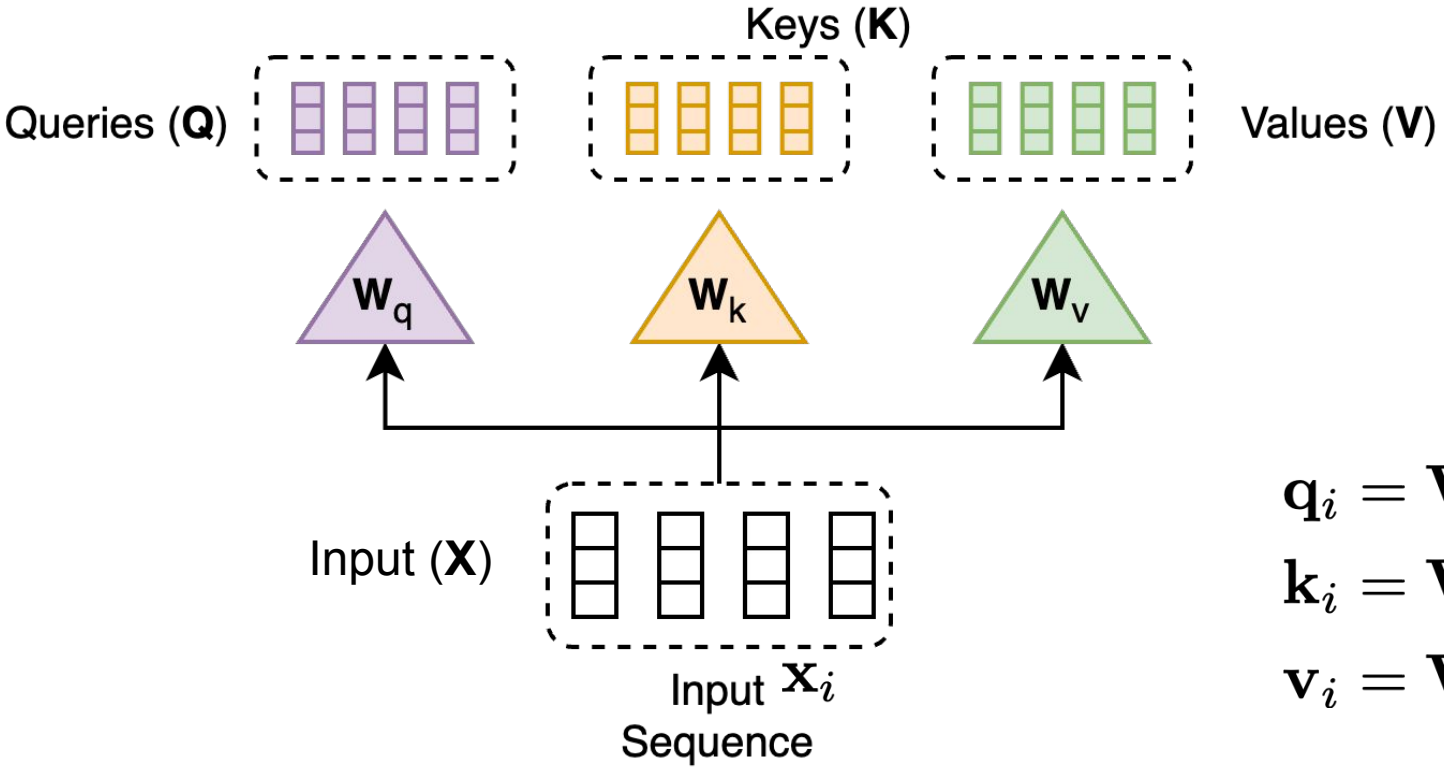
Transformer Architecture



Transformer Architecture



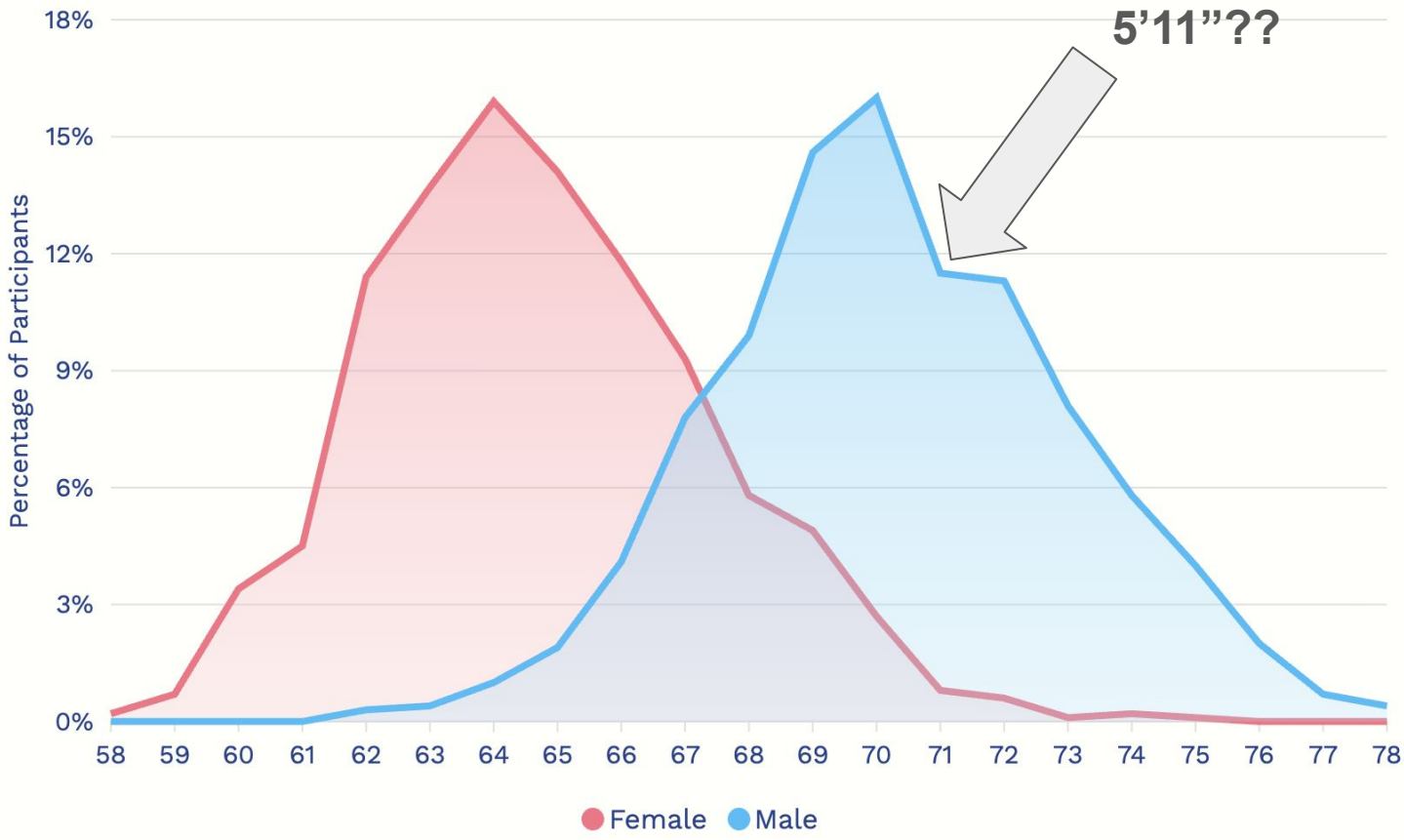
Self-Attention



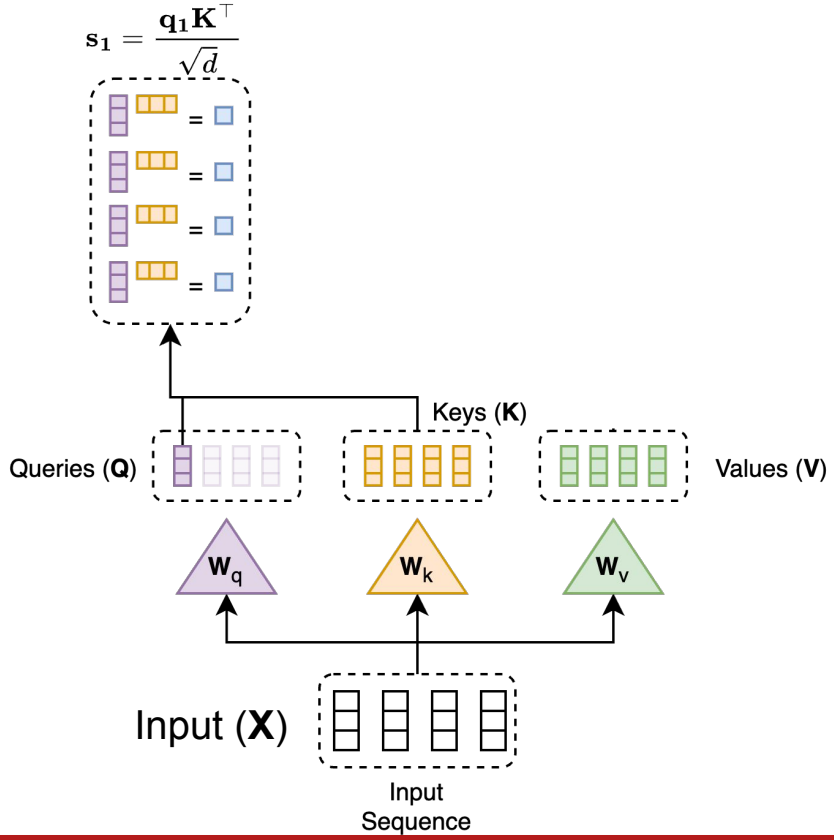
$$\mathbf{q}_i = \mathbf{W}_q \mathbf{x}_i$$
$$\mathbf{k}_i = \mathbf{W}_k \mathbf{x}_i$$
$$\mathbf{v}_i = \mathbf{W}_v \mathbf{x}_i$$

Cornell

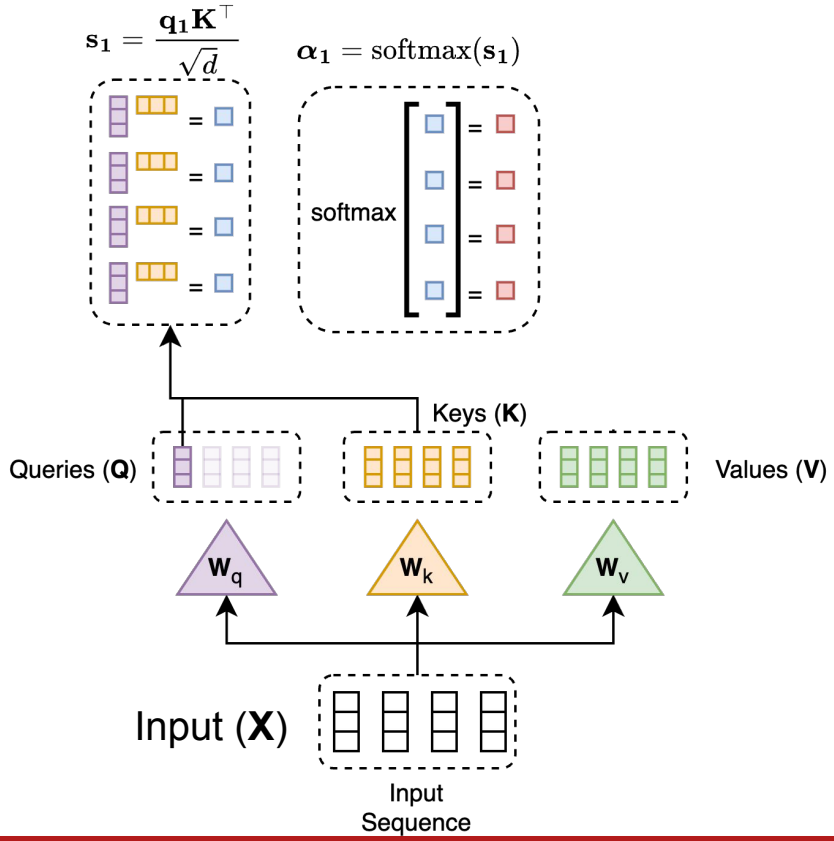
Viz by Gender: **Your height?**



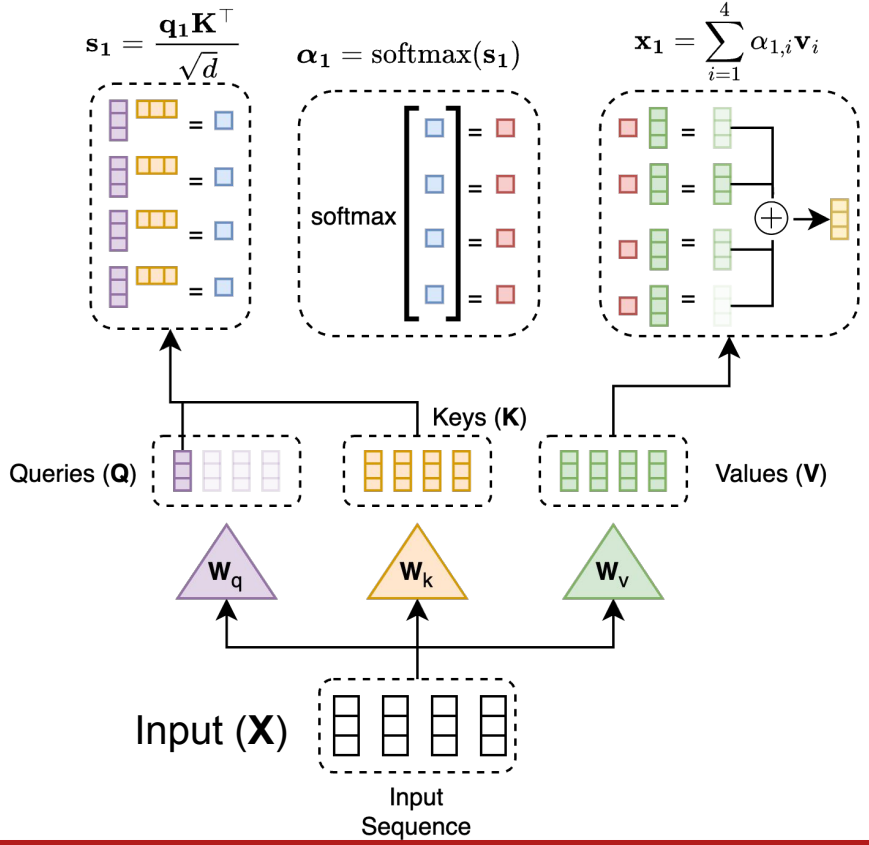
Self-Attention



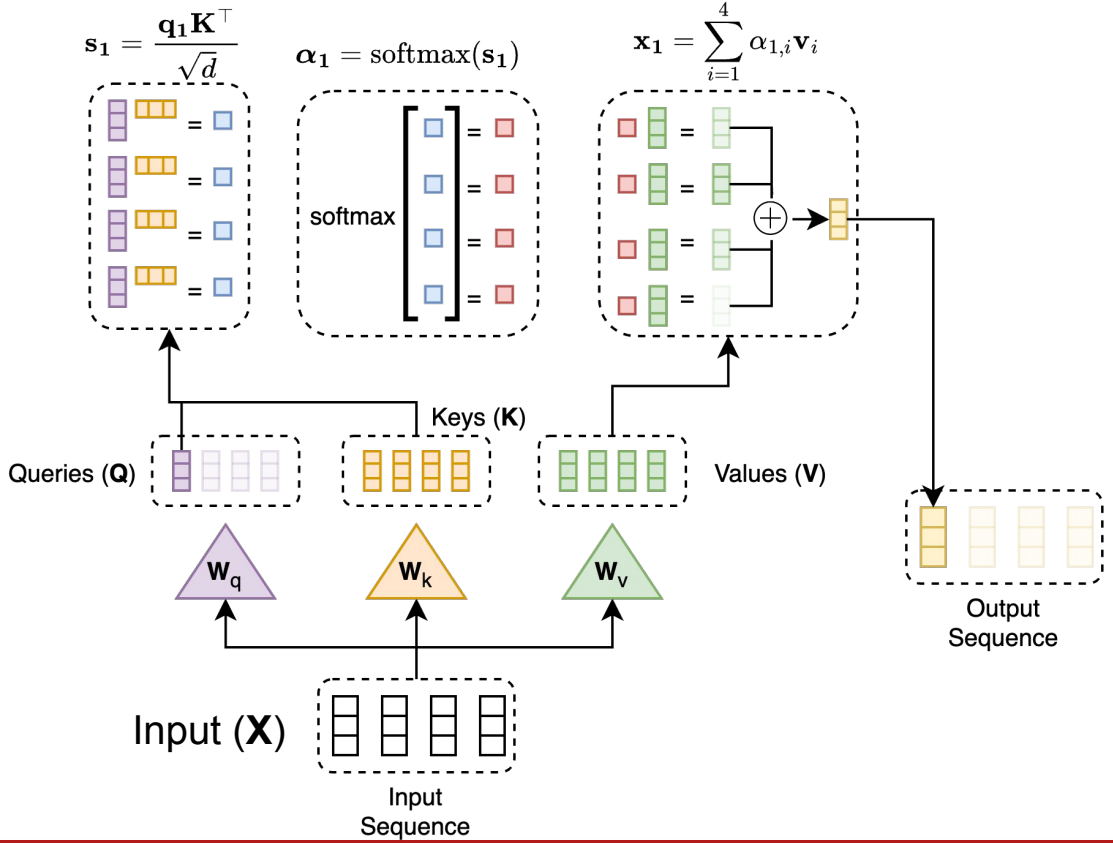
Self-Attention



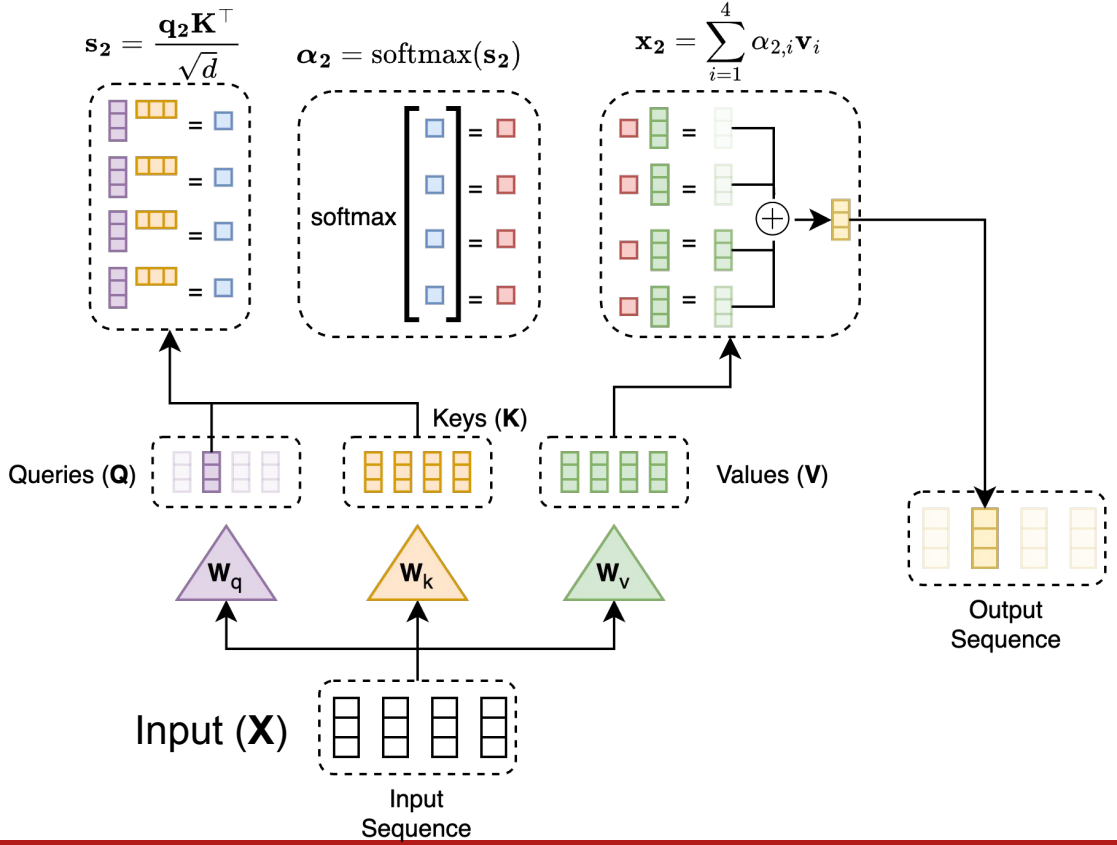
Self-Attention



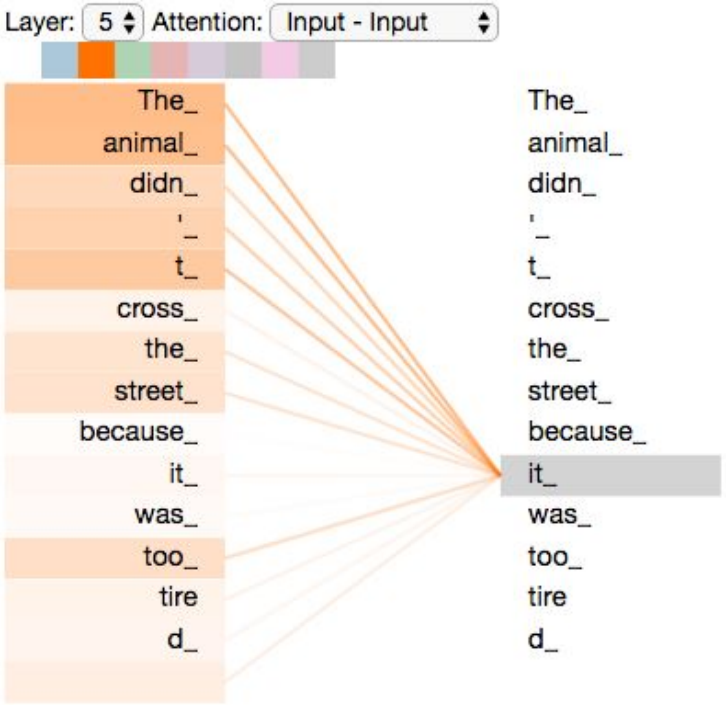
Self-Attention



Self-Attention



Self-attention demo



Discuss:

- Q, K, V are all (n x d) matrices. Consider have an input of shape b x n x d.

- What is the shape of QK^T?
  - What does this matrix represent?
- What is the shape of the final output?
  - What does this matrix represent?

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Multi-Head Attention

What if I want to pay attention to different things at the same time!?

Content-based	This is my big red dog, Clifford.
Description-based	This is my big red dog, Clifford.
Reference-based	This is my big red dog, Clifford.

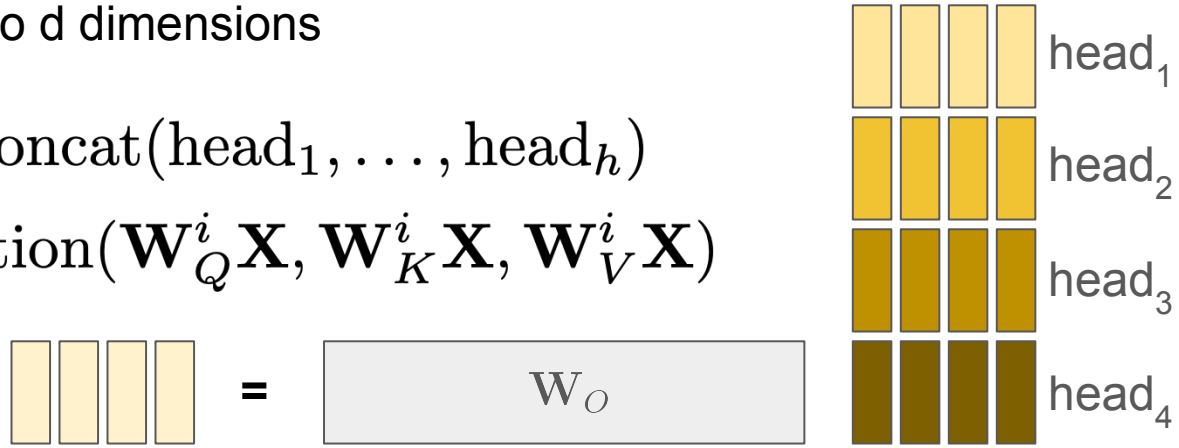
What's useful depends on the task. How do I pick what to do?

Multi-Head Attention

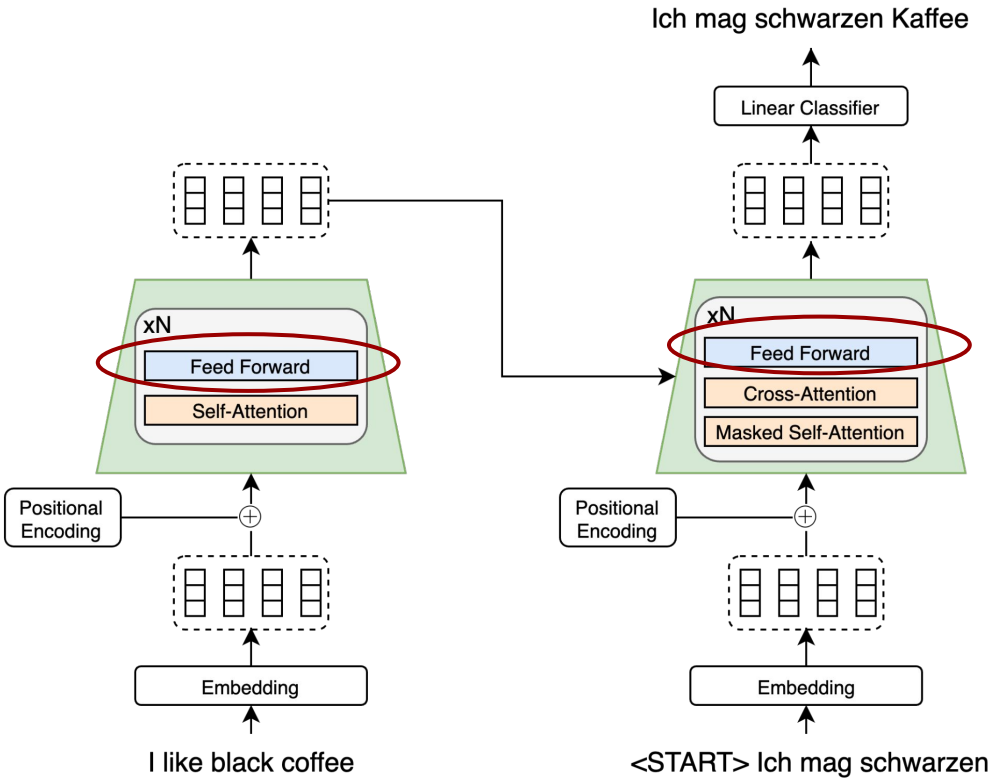
- The Scaled Dot-Product Attention attends to one or few entries in the input key-value pairs.
- Idea: apply Scaled Dot-Product Attention multiple times on the linearly transformed inputs.
- Concatenate outputs
- Project them down into d dimensions

$$\text{MultiHead}(\mathbf{X}) = \mathbf{W}_O \text{Concat}(\text{head}_1, \ldots, \text{head}_h)$$

$$\text{where head}_i = \text{Attention}(\mathbf{W}_Q^i \mathbf{X}, \mathbf{W}_K^i \mathbf{X}, \mathbf{W}_V^i \mathbf{X})$$

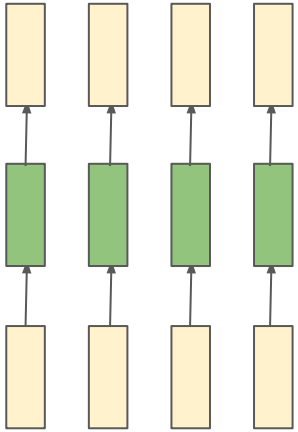


Transformer Architecture



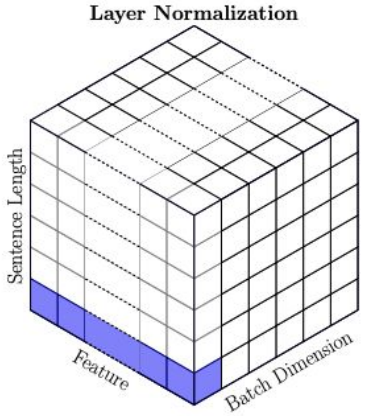
Point-wise Feed-forward Networks

- Apply MLP (aka FFN) to each output vector
  - The **same** MLP to each vector
  - Completely **parallel**
  - $\mathrm{FFN}(\mathbf{x}) = \max(0, \mathbf{W}\mathbf{x} + b)$
  - where **W** and **b** are learned weights and biases

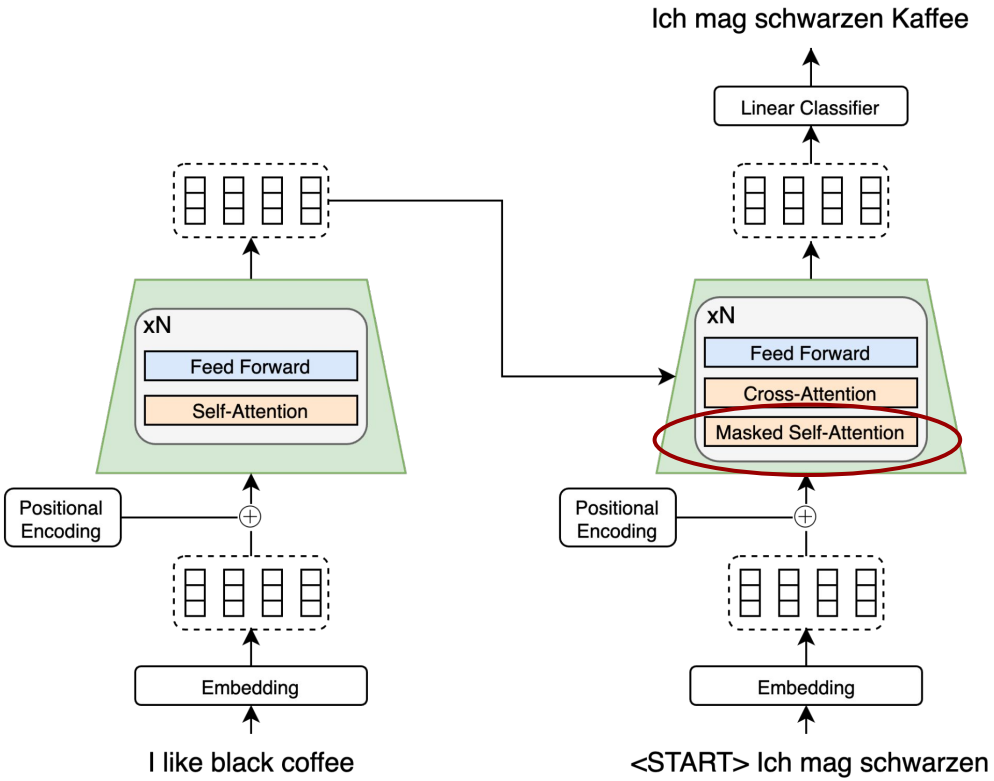


Layernorm

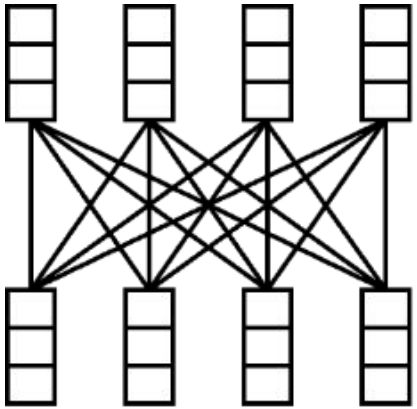
- The FFN and Self-Attention layers are followed by **Layer Normalization**.
- Same as Batch Norm, except:
  - Each word vector is normalized independently



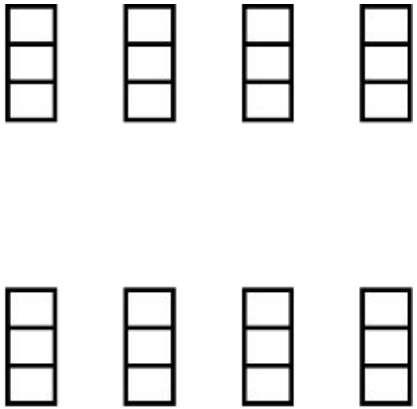
Transformer Architecture



Self-Attention vs. Masked Self-Attention

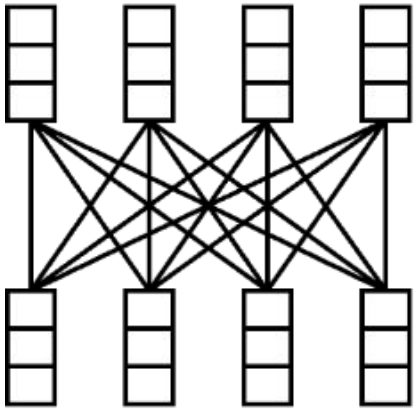


Self-Attention

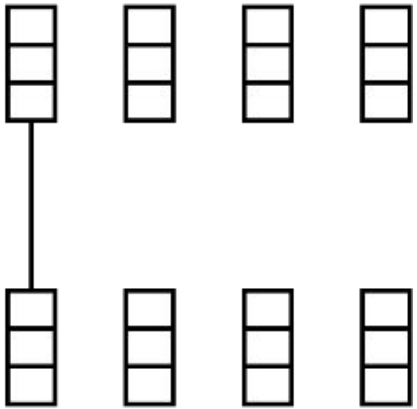


Masked Self-Attention

Self-Attention vs. Masked Self-Attention

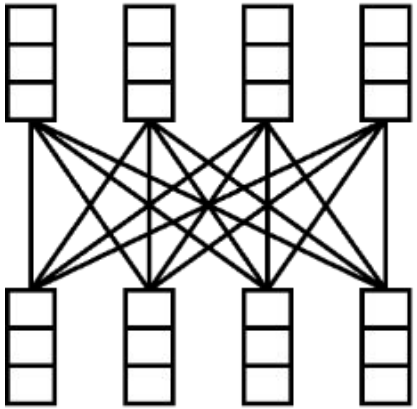


Self-Attention

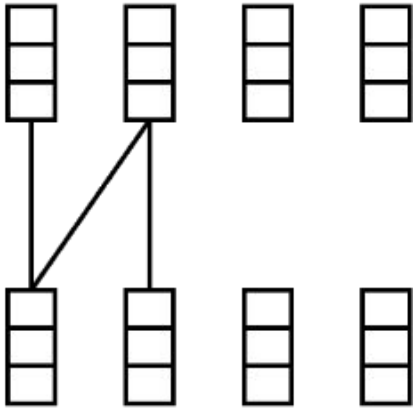


Masked Self-Attention

Self-Attention vs. Masked Self-Attention

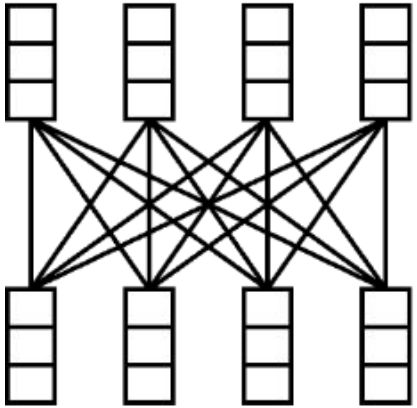


Self-Attention

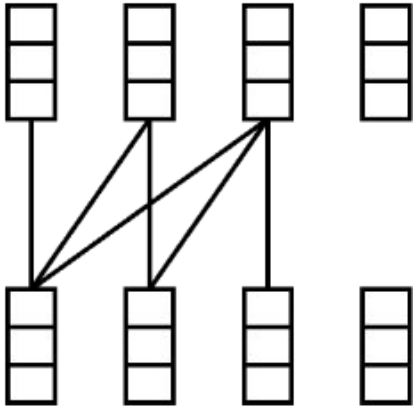


Masked Self-Attention

Self-Attention vs. Masked Self-Attention

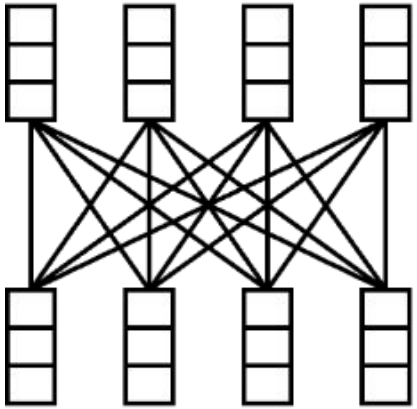


Self-Attention

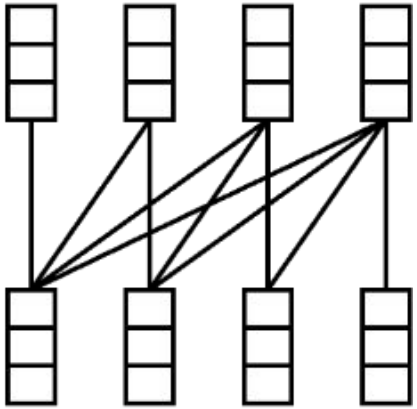


Masked Self-Attention

Self-Attention vs. Masked Self-Attention

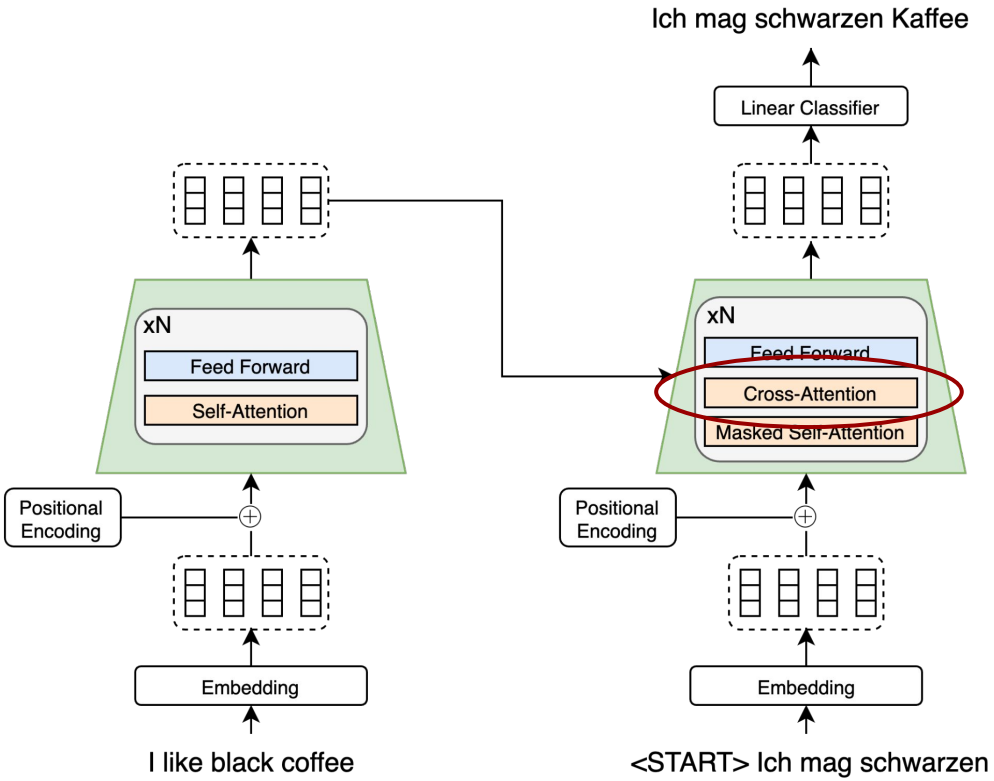


Self-Attention

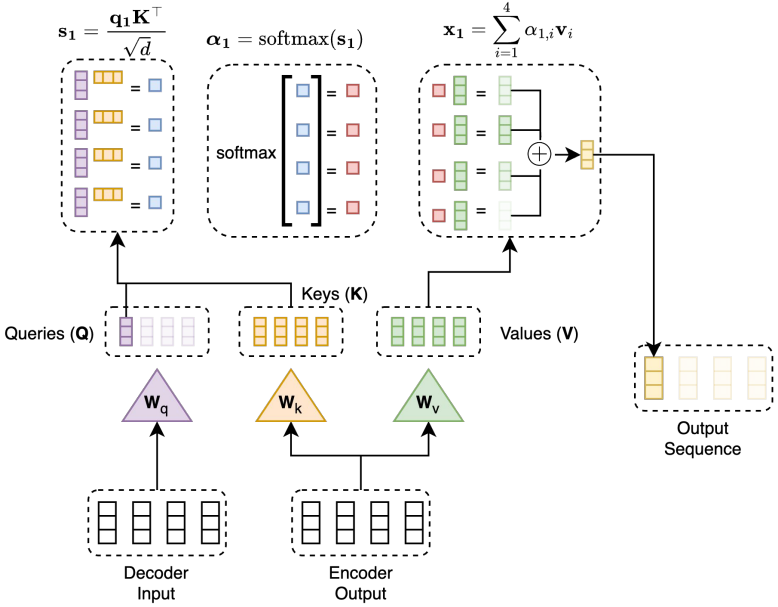


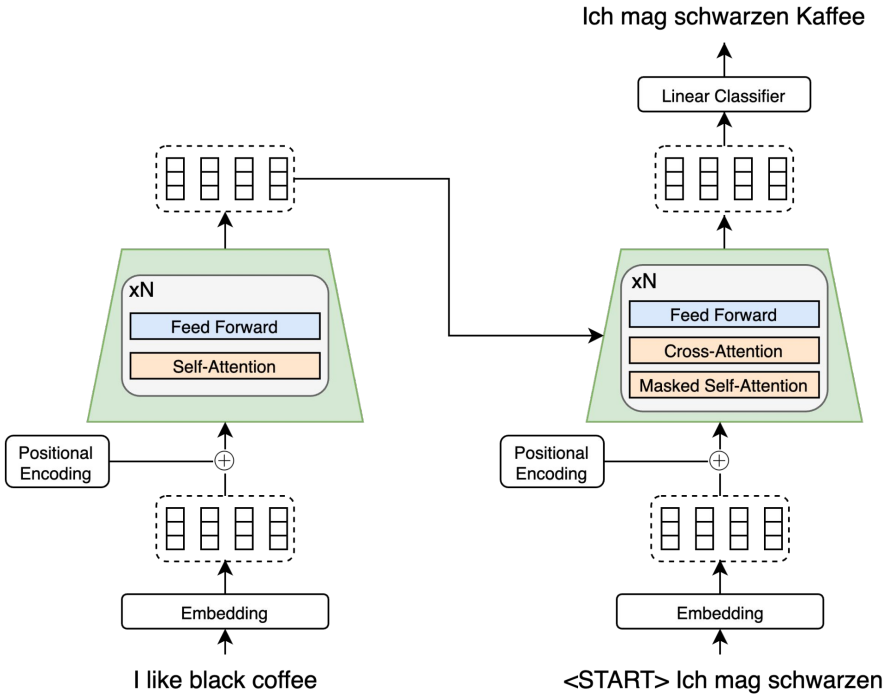
Masked Self-Attention

Transformer Architecture

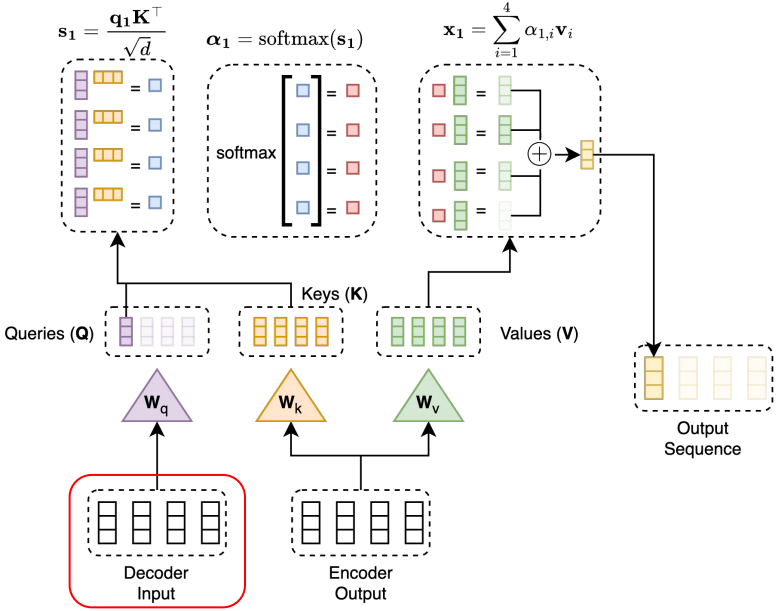


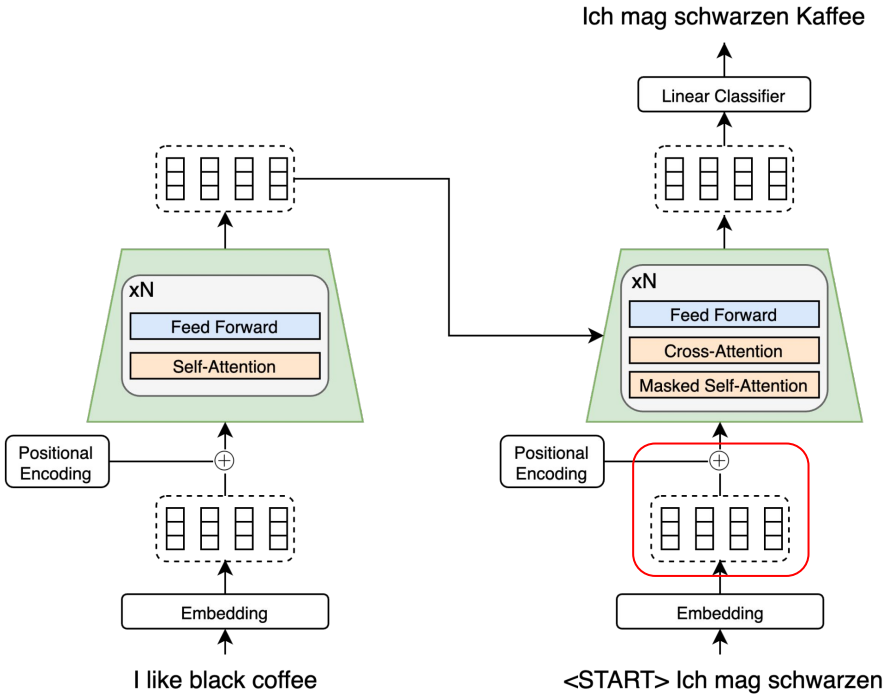
Cross-Attention



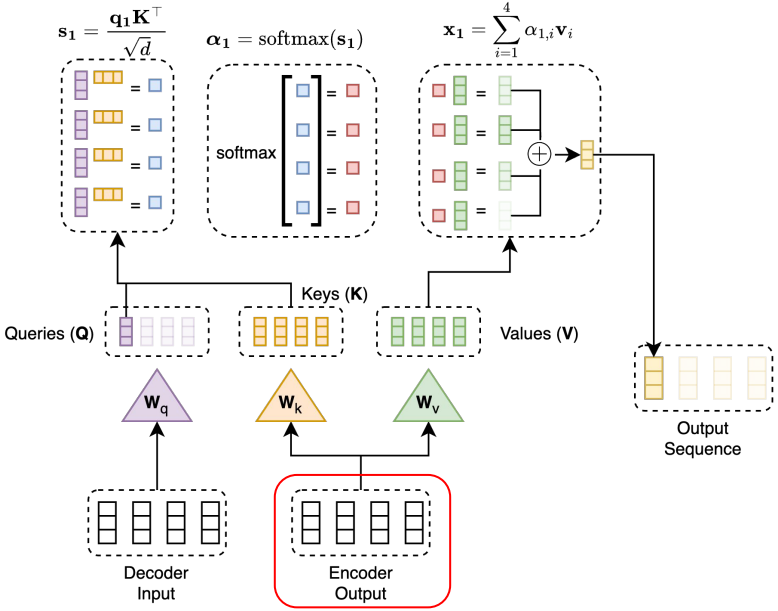


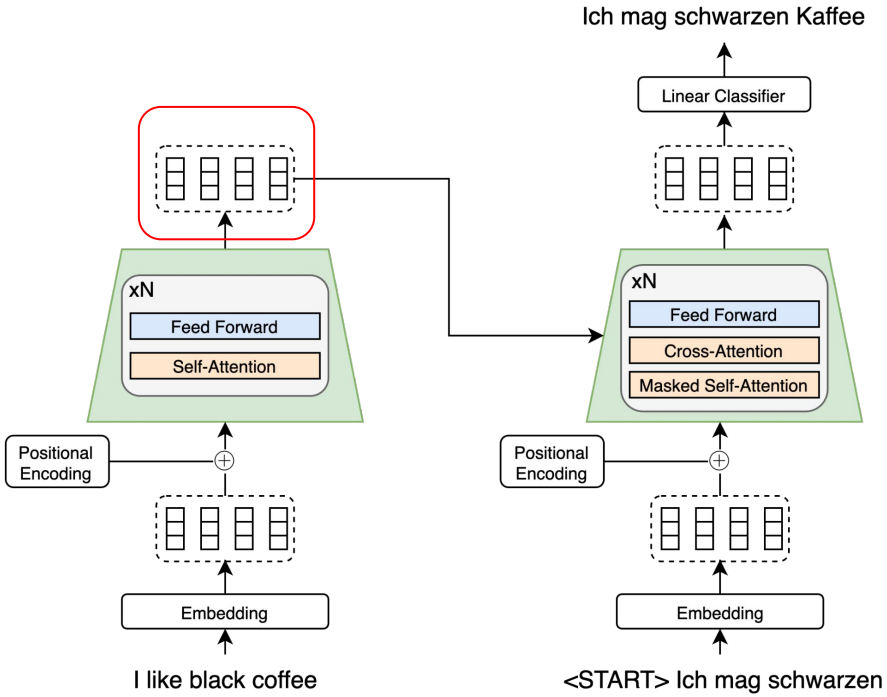
Cross-Attention



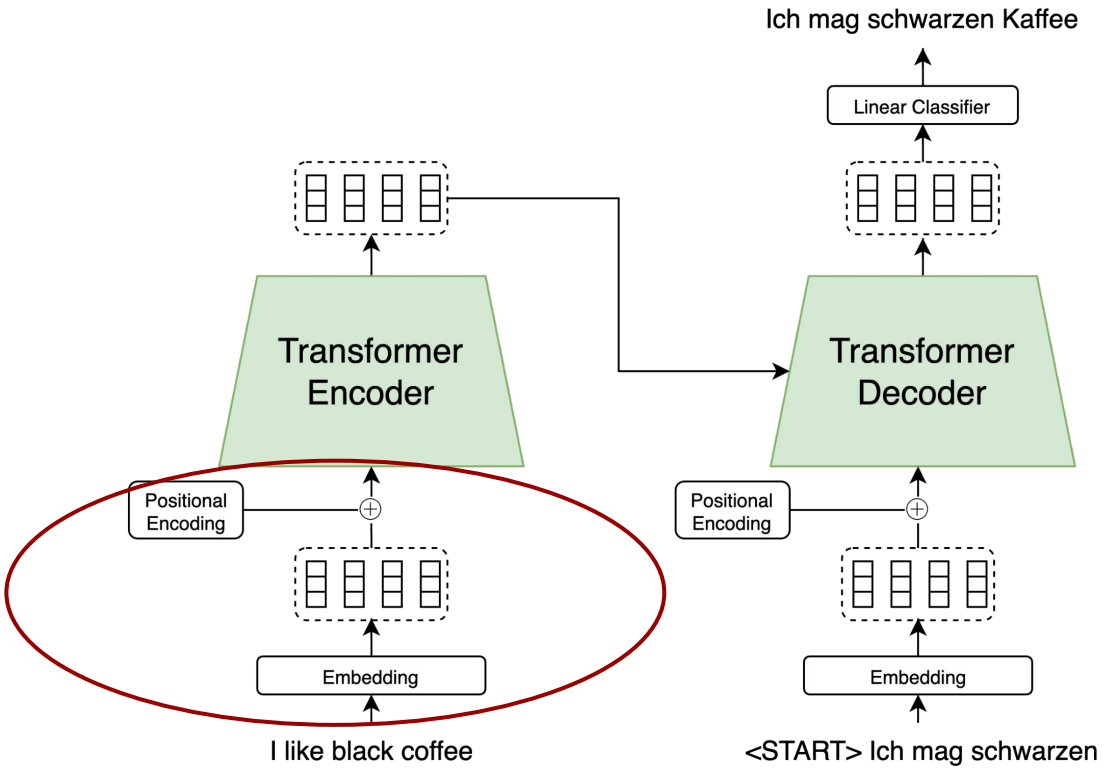


Cross-Attention





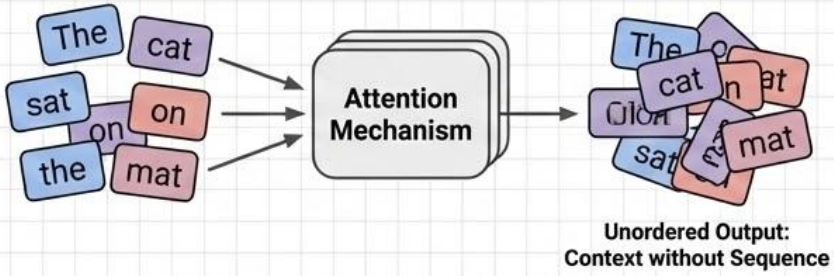
Transformer Architecture



Understanding Positional Embeddings

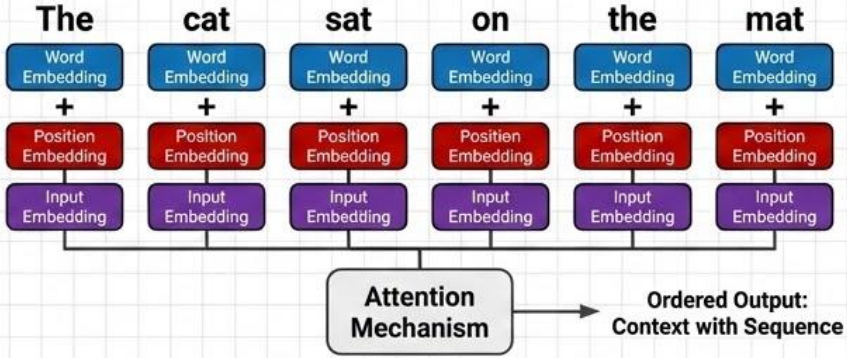
Giving Order to Sequence Data in Transformers

The Problem: Attention is Permutation-Invariant



Self-attention processes all tokens simultaneously, losing their original sequence order. "The cat sat on the mat" is treated the same as "Mat the on sat cat the".

The Solution: Injecting Position Information



Positional embeddings are added to the input word embeddings to provide information about the token's position. This allows the model to distinguish between the same word in different positions.

Types of Positional Embeddings

Learned

Learned: The model learns a unique vector for each position during training.

Sinusoidal (Fixed)

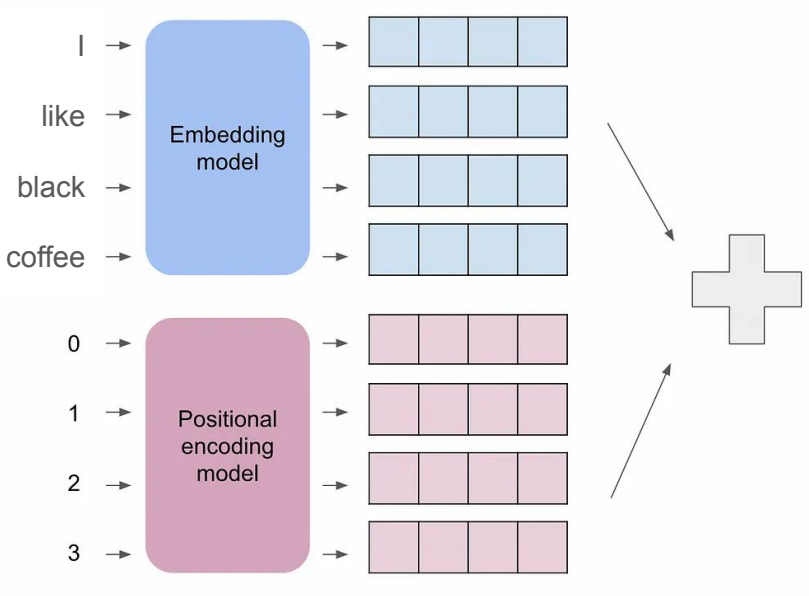
Sinusoidal: Uses fixed sine and cosine functions to encode positions, allowing generalization to longer sequences.

Relative

Relative: Encodes the relative distance between tokens, rather than absolute positions.

Positional Encoding

- Embedding model:
  - Lookup table
  - Non-contextual word embeddings
- Problem:
  - Word order matters!
  - In attention word order is lost
- Solution:
  - Encode the **token position** as a vector
  - Add this vector to the **word embedding**



[https://medium.com/@xuer.chen.human/llm-study-notes-positional-encoding-0639a1002ec0]

Positional Encoding

Neural networks are not good with single dimensional values.
Higher dimensions are better suited for hyper-plane based classifiers (internal to the neural network).
Naive Idea: Use binary encoding

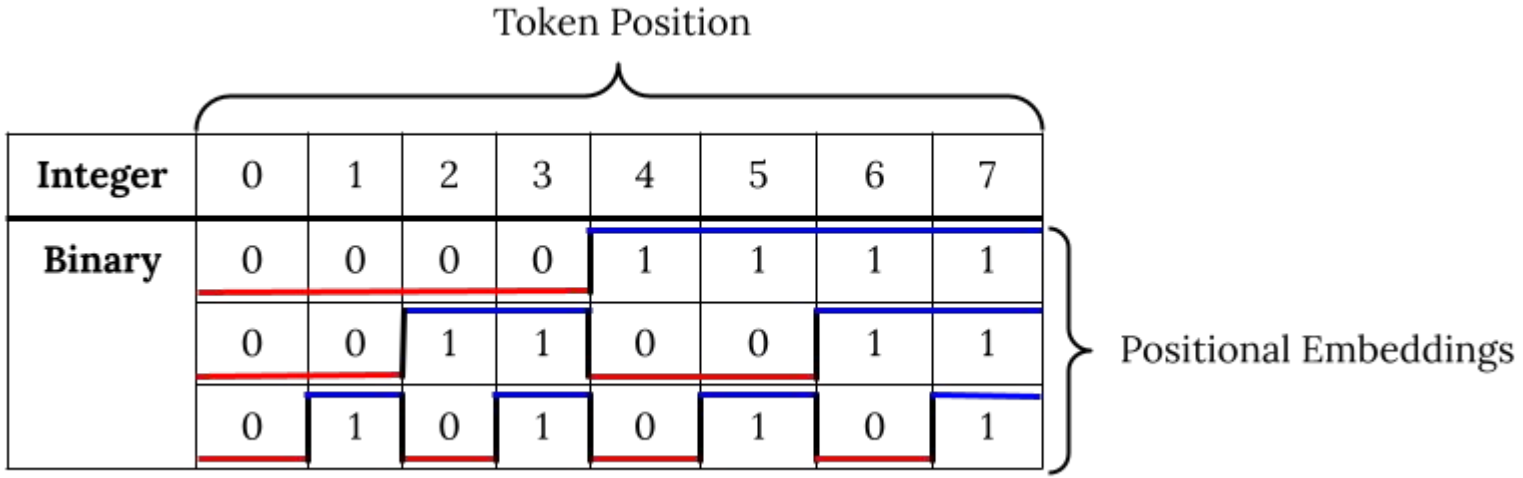
Token Position

Integer	0	1	2	3	4	5	6	7
Binary	0	0	0	0	1	1	1	1
	0	0	1	1	0	0	1	1
	0	1	0	1	0	1	0	1

Positional Embeddings

Positional Encoding

Neural networks are not good with single dimensional values.
Higher dimensions are better suited for hyper-plane based classifiers (internal to the neural network).
Naive Idea: Use binary encoding

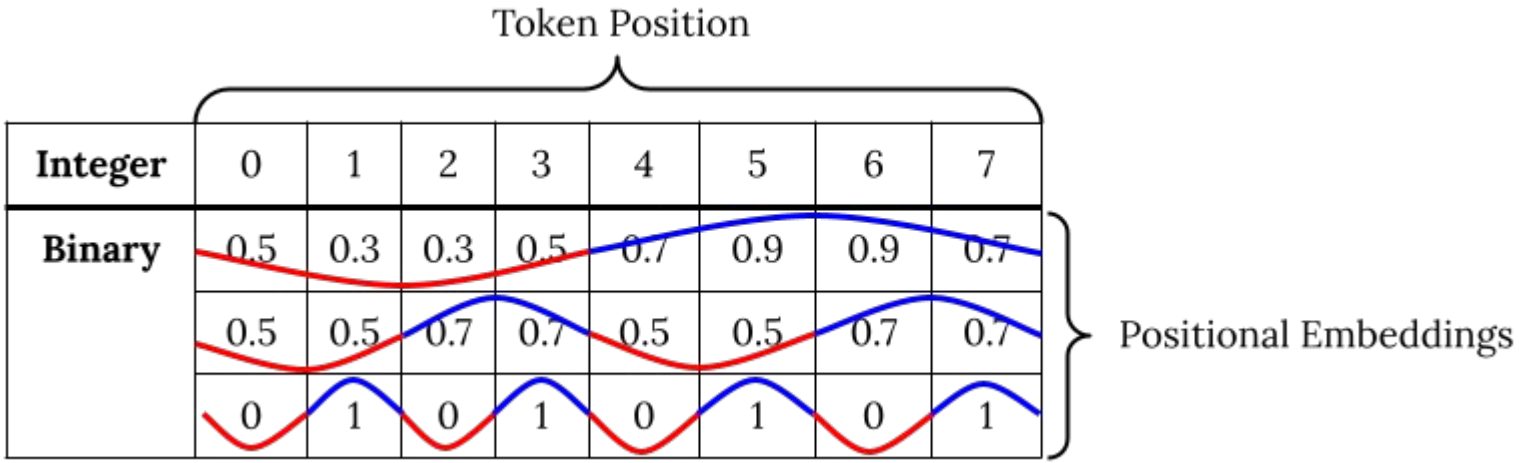


Positional Encoding

Better idea: Use cosine / sine embeddings (Fourier features.)

For a position pos and embedding dimension i:

- $PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$
- $PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$

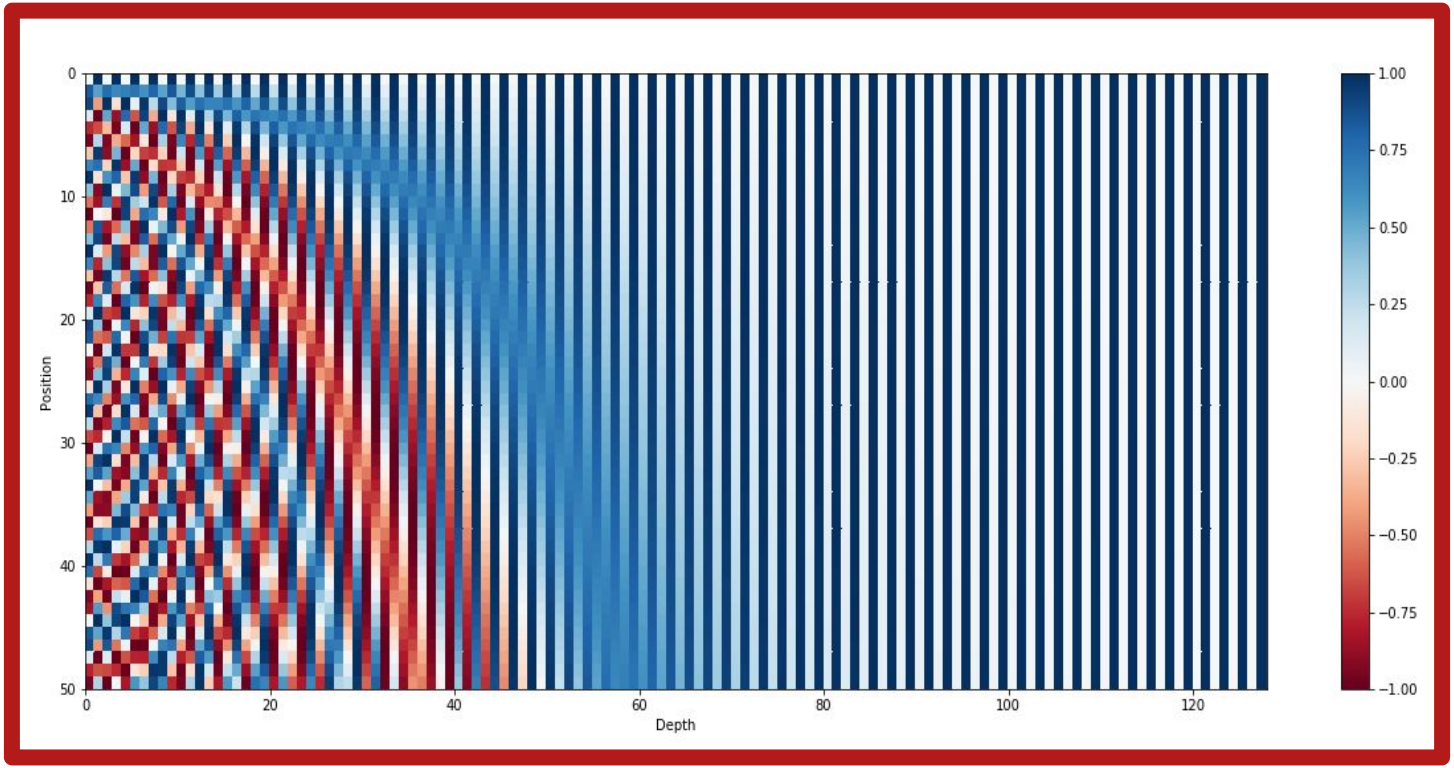


Cornell Bowers CIS

Example Positional Encoding

For a position pos and embedding dimension i:

- $PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$
- $PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$

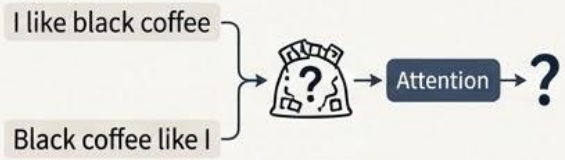


Cornell Bowers CIS

Positional Embeddings: Encoding Order in Transformers

1. THE PROBLEM: ORDER MATTERS

Attention mechanisms process words in parallel, losing sequence information. We need a way to re-introduce it.



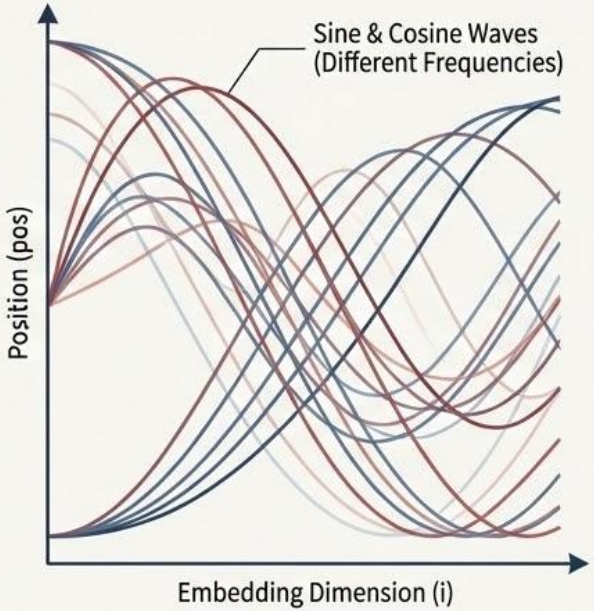
THE CONCEPT: ADDING POSITION

Encode each token's position as a vector and add it to its semantic embedding.

[Word Embedding] + [Position Vector] = [Contextualized Input]

2. THE SOLUTION: FOURIER FEATURES

Simple integers are not ideal for neural networks. We use sine and cosine functions to create smooth, continuous embeddings across dimensions.



3. FORMULA & VISUALIZATION

THE FORMULA

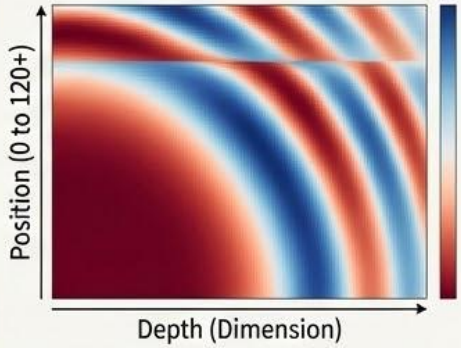
For position 'pos' and dimension 'i':

$$PE(pos, 2i) = \sin(pos / 10000^{2i/d_{model}})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{2i/d_{model}})$$

Even dimensions use sine, odd use cosine.

THE EMBEDDING HEATMAP

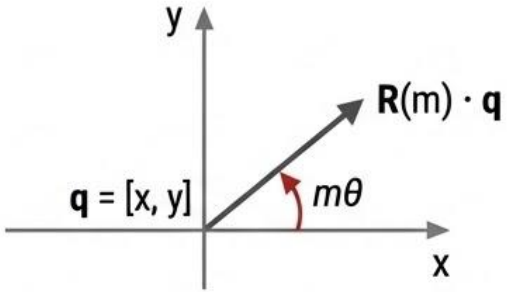


Each row (position) has a unique pattern across dimensions (depth), allowing the model to distinguish between positions.

Cornell Bowers CIS

RoPE: Rotary Position Embedding

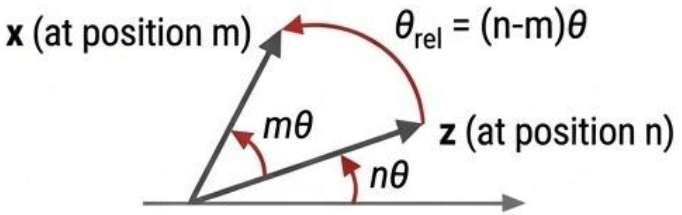
The Core Rotation Mechanism



$$\mathbf{R}(m)\cdot\mathbf{q} = \begin{bmatrix} x\cdot\cos(m\theta) - y\cdot\sin(m\theta) \\ x\cdot\sin(m\theta) + y\cdot\cos(m\theta) \end{bmatrix}$$

A 2D rotation matrix **R**(*m*) is applied to a query vector **q**. The rotation angle $m\theta$ is proportional to the position *m*.

Preserving Relative Position



Rotation is **additive**: $\mathbf{R}(m)\mathbf{R}(n)=\mathbf{R}(m+n)$ and $\mathbf{R}(m)^T\mathbf{R}(n)=\mathbf{R}(m-n)$

$$\langle \mathbf{x},\mathbf{z} \rangle = (\mathbf{R}(m)\cdot\mathbf{x})^T\,(\mathbf{R}(n)\cdot\mathbf{z}) = \mathbf{x}^T\,\mathbf{R}(m-n)\,\mathbf{z}$$

For vectors **x** at position *m* and **z** at position *n*, the dot product $\langle \mathbf{x},\mathbf{z} \rangle$ depends only on the relative distance (*n*-*m*).

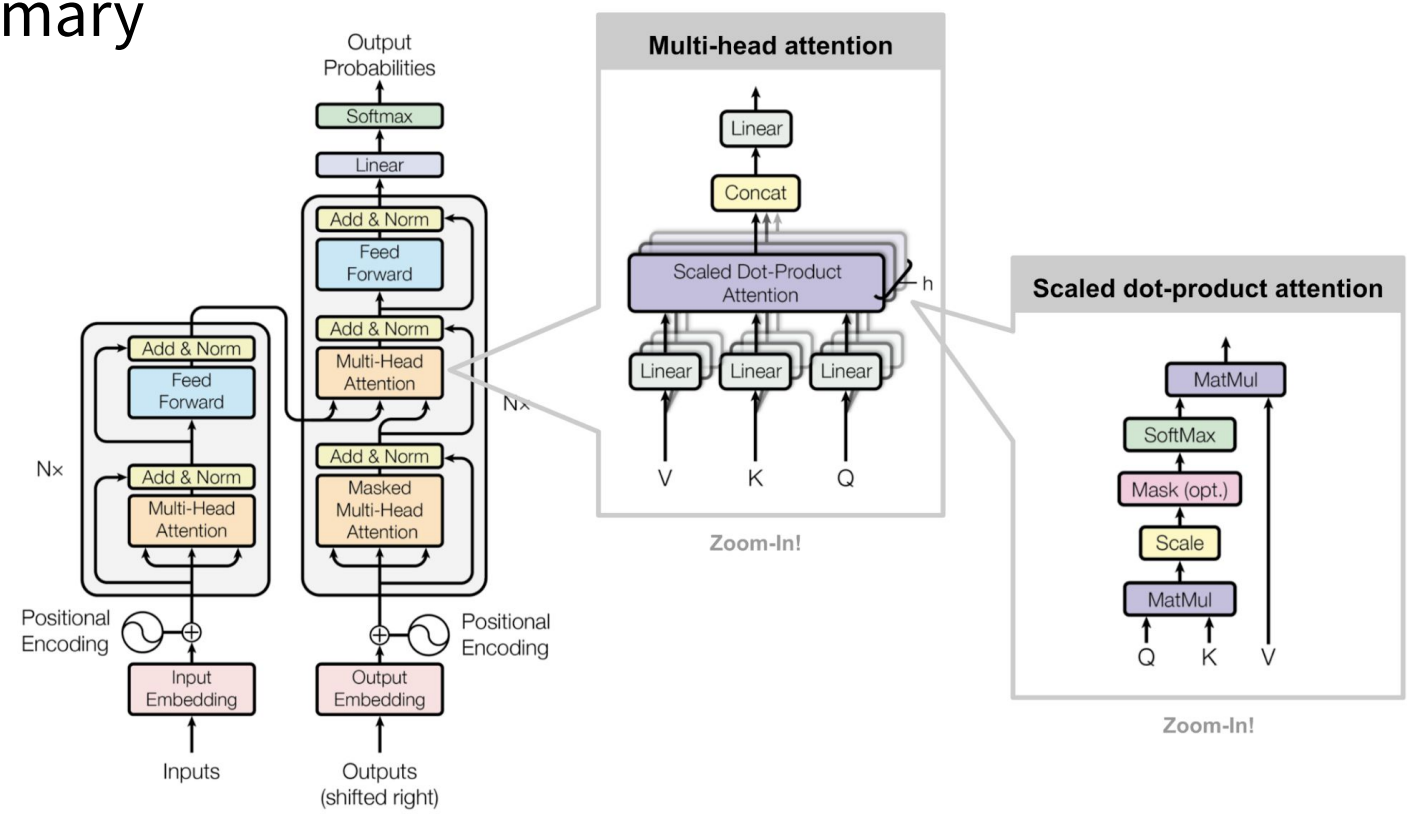
Generalization to Higher Dimensions

$$[x_1, x_{2i}, x_{2i+1}, \ldots, x_d]$$

For higher dimensional vectors, apply the 2D rotation to pairs of dimensions.

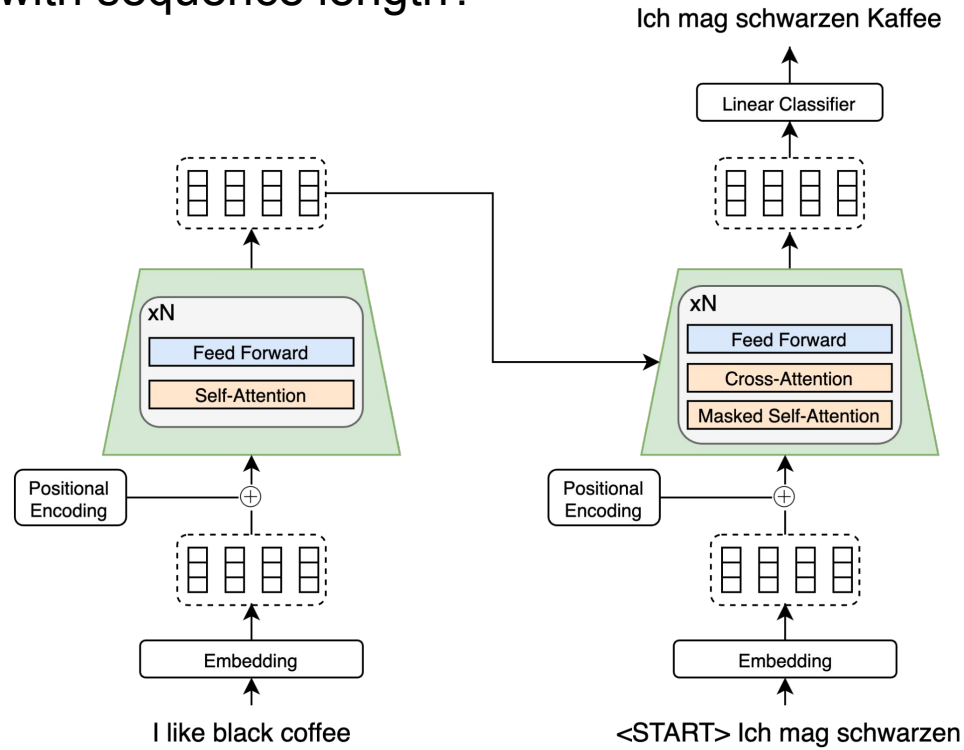
Define the angle of position *i*/2 as $\theta_i = \frac{1}{10000^{2i/d}}$.

Summary


Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)
Multi-head attention
Linear
Concat
Scaled Dot-Product Attention
h
Linear
Linear
Linear
V
K
Q
Zoom-In!
Scaled dot-product attention
MatMul
SoftMax
Mask (opt.)
Scale
MatMul
Q
K
V
Zoom-In!

Discuss:

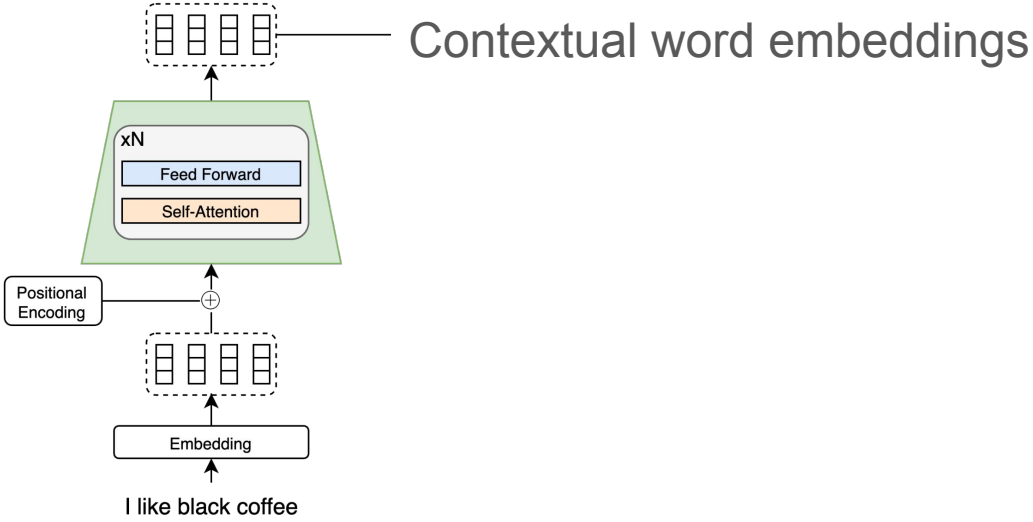
- How does the transformer scale with sequence length?



Interactive Transformer Demo

BERT (Bidirectional Encoder Representations from Transformers)

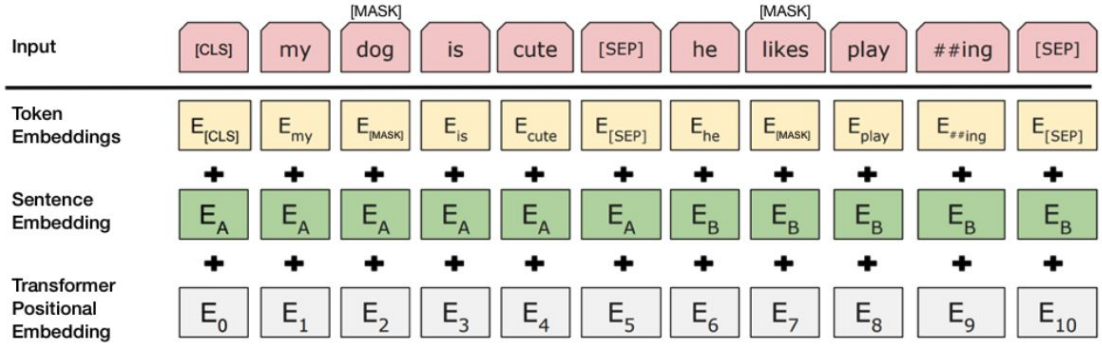
- Output of **Transformer Encoder** as contextual word embeddings
- Bidirectional Context
- Pre-trained on the language, and then fine-tuned



BERT - Input Representation

Input:

- Use 30,000 WordPiece vocabulary on input.
- Each token is sum of three embeddings



Training

- Masked Language Modelling
  - Mask out k% of the input words, and then predict the masked words
  - the man went to the store to [MASK] a [MASK] of milk
  - What can you use as a loss function?

- Next sentence prediction
  - To learn relationships between sentences, predict whether Sentence B is actual sentence that
  - proceeds Sentence A, or a random sentence

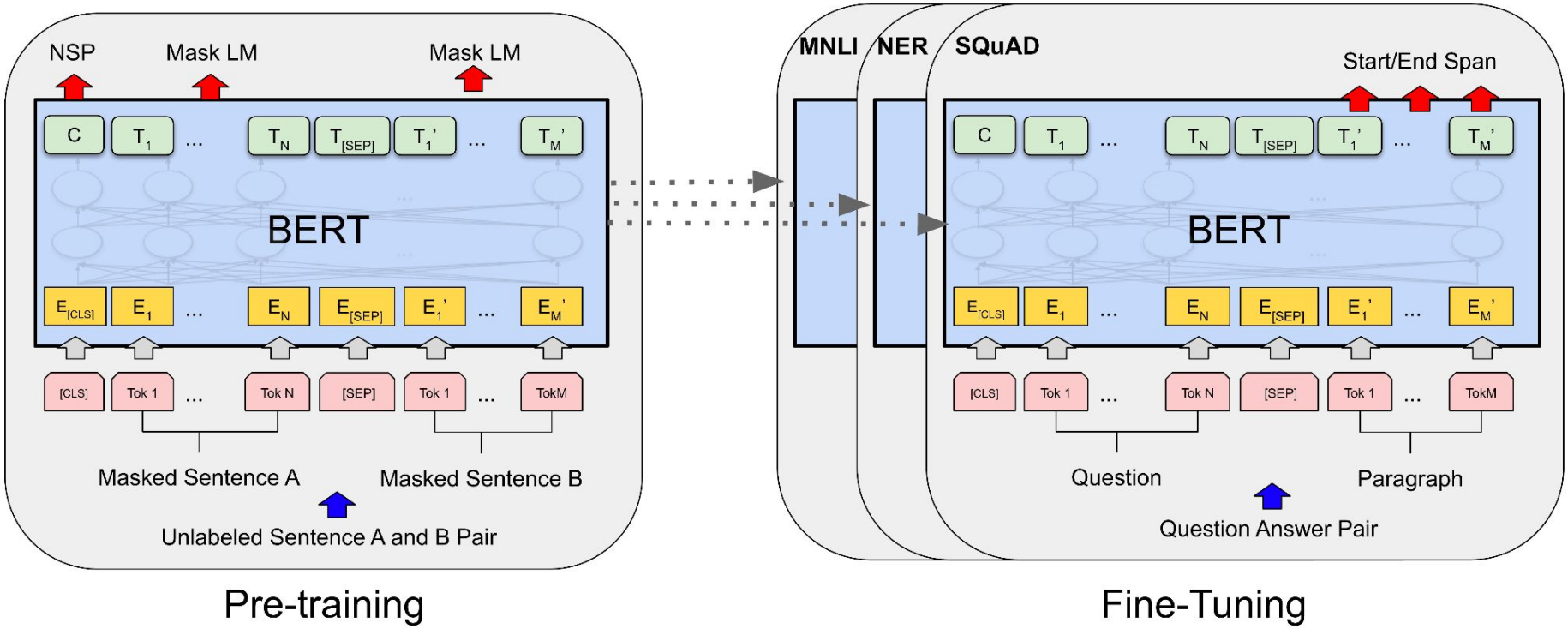
Sentence A = `The man went to the store.`
Sentence B = `He bought a gallon of milk.`
Label = `IsNextSentence`

Sentence A = `The man went to the store.`
Sentence B = `Penguins are flightless.`
Label = `NotNextSentence`

Model Details

- Data: Wikipedia (2.5B words) + BookCorpus (800M words)
- Batch Size: 131,072 words (1024 sequences * 128 length or 256 sequences * 512 length)
- Training Time: 1M steps (~40 epochs)
- Optimizer: AdamW, 1e-4 learning rate, linear decay
- BERT-Base: 12-layer, 768-hidden, 12-head, 110M params
- BERT-Large: 24-layer, 1024-hidden, 16-head, 340M params
- Trained on 4x4 or 8x8 TPU slice for 4 days

Pre-training to Fine-tuning Pipeline



System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

https://arxiv.org/pdf/1810.04805.pdf

Self-supervised Learning

- Labels are generated automatically, no human labeling process
- Benefits
  - Scales well
  - Cost-Efficient
  - Flexible
- Challenges
  - Larger datasets are required
  - More compute is necessary

Review

- LSTMs/GRUs are recurrent
- Self-attention can effectively replace recurrence in sequence-to-sequence models
- Transformers use self-attention and are parallelizable
- Pre-training using self-supervised learning help train large models that learn very good representations